Experiment 'p_14_7_bubble_sort_array' Results

June 5, 2026

Experiment outcome: FAILURE_RESOLVE
Bad responses: 1
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 3
Soft fails (resolution): 0
Verification attempts: 0

Problem Specification

Problem name: p_14_7_bubble_sort_array
Natural language statement: null
Method signature: p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures
    ↪ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQFvvDUEu0udUCUyY51CmFZThqBB
Model: gpt-5.5-2026-04-23
Created at: 1780671371
Finish reason: stop
Usage: 310 tokens in, and 8355 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
```

```
  invariant 1 <= j <= i
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
  invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
  decreases i - j
{
  ghost var before := sorted[..];
  assert before == sorted[..];
  assert before.Length == n;
  assert multiset(before) == multiset(inputs[..]);
  assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
  assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

  if before[j] < before[j - 1] {
    var t := sorted[j - 1];
    sorted[j - 1] := sorted[j];
    sorted[j] := t;

    assert sorted[j - 1] == before[j];
    assert sorted[j] == before[j - 1];

    ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
        ↪ 1..];
    assert swapped.Length == before.Length;
    assert forall k | 0 <= k < sorted.Length
      ensures sorted[k] == swapped[k]
    {
      if k < j - 1 {
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert swapped[k] == before[j];
      } else if k == j {
        assert sorted[k] == before[j - 1];
        assert swapped[k] == before[j - 1];
      } else {
        assert j + 1 <= k < sorted.Length;
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      }
    }
    assert sorted[..].Length == swapped.Length;
    assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
    assert sorted[..] == swapped;

    calc {
      multiset(sorted[..]);
```

```
    ==
      multiset(swapped);
    ==
      multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
          ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
          ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
    == {
      assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
          ↪ 1..];
    }
      multiset(before);
    }
  } else {
    assert sorted[..] == before;
    assert multiset(sorted[..]) == multiset(before);
  }

  assert multiset(sorted[..]) == multiset(inputs[..]);

  assert forall p, q | i <= p < q < n
    ensures sorted[p] <= sorted[q]
  {
    assert sorted[p] == before[p];
    assert sorted[q] == before[q];
    assert before[p] <= before[q];
  }

  assert forall p, q | 0 <= p < i <= q < n
    ensures sorted[p] <= sorted[q]
  {
    if before[j] < before[j - 1] {
      if p == j - 1 {
        assert sorted[p] == before[j];
        assert before[j] <= before[q];
      } else if p == j {
        assert sorted[p] == before[j - 1];
        assert before[j - 1] <= before[q];
      } else {
        assert sorted[p] == before[p];
        assert before[p] <= before[q];
      }
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
```

```
    assert sorted[q] == before[q];
  }

  assert forall k | 0 <= k < j + 1
    ensures sorted[k] <= sorted[j]
  {
    if before[j] < before[j - 1] {
      assert sorted[j] == before[j - 1];
      if k == j {
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert before[j] < before[j - 1];
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
      }
    } else {
      assert before[j - 1] <= before[j];
      assert sorted[j] == before[j];
      if k == j {
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
        assert before[k] <= before[j];
      }
    }
  }

  j := j + 1;
}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

assert forall p, q | i - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == i - 1 {
    if q < i {
      assert q == i - 1;
    } else {
      assert i <= q < n;
      assert 0 <= i - 1 < i <= q < n;
      assert sorted[i - 1] <= sorted[q];
    }
  } else {
    assert i <= p < q < n;
    assert sorted[p] <= sorted[q];
  }
```

```
    }

    assert forall p, q | 0 <= p < i - 1 <= q < n
      ensures sorted[p] <= sorted[q]
    {
      if q == i - 1 {
        assert 0 <= p < i;
        assert sorted[p] <= sorted[i - 1];
      } else {
        assert i <= q < n;
        assert 0 <= p < i <= q < n;
        assert sorted[p] <= sorted[q];
      }
    }

    i := i - 1;
  }

  assert i == 0;
  assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_7_bubble_sort_array/program.dfy(89,6): Error: invalid UnaryExpression
   |
89 | } else {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(88,8): Error: expected either 'by' or a
    ↪  semicolon following the assert expression
   |
88 | }
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(152,8): Error: invalid AssertStmt
    |
152 | j := j + 1;
    | ^

3 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQIitSQ3LH9guG4fzL4tOuNArKBn
Model: gpt-5.5-2026-04-23
Created at: 1780671544
Finish reason: stop
Usage: 2504 tokens in, and 3363 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
      decreases i - j
    {
      ghost var before := sorted[..];
      assert before == sorted[..];
      assert before.Length == n;
      assert multiset(before) == multiset(inputs[..]);
      assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
      assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
      assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

      if before[j] < before[j - 1] {
        var t := sorted[j - 1];
        sorted[j - 1] := sorted[j];
        sorted[j] := t;

        assert sorted[j - 1] == before[j];
        assert sorted[j] == before[j - 1];
```

```
  ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
      ↪ 1..];
  assert swapped.Length == before.Length;
  assert forall k | 0 <= k < sorted.Length :: sorted[k] == swapped[k] by {
    if k < j - 1 {
      assert sorted[k] == before[k];
      assert swapped[k] == before[k];
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert swapped[k] == before[j];
    } else if k == j {
      assert sorted[k] == before[j - 1];
      assert swapped[k] == before[j - 1];
    } else {
      assert j + 1 <= k < sorted.Length;
      assert sorted[k] == before[k];
      assert swapped[k] == before[k];
    }
  }
  assert sorted[..].Length == swapped.Length;
  assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
  assert sorted[..] == swapped;

  calc {
    multiset(sorted[..]);
  ==
    multiset(swapped);
  ==
    multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
  ==
    multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
        ↪ multiset(before[j + 1..]);
  ==
    multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
        ↪ multiset(before[j + 1..]);
  ==
    multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
  == {
    assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
        ↪ 1..];
  }
    multiset(before);
  }
} else {
  assert sorted[..] == before;
  assert multiset(sorted[..]) == multiset(before);
}

assert multiset(sorted[..]) == multiset(inputs[..]);
```

```
assert forall p, q | i <= p < q < n :: sorted[p] <= sorted[q] by {
  assert sorted[p] == before[p];
  assert sorted[q] == before[q];
  assert before[p] <= before[q];
}

assert forall p, q | 0 <= p < i <= q < n :: sorted[p] <= sorted[q] by {
  if before[j] < before[j - 1] {
    if p == j - 1 {
      assert sorted[p] == before[j];
      assert before[j] <= before[q];
    } else if p == j {
      assert sorted[p] == before[j - 1];
      assert before[j - 1] <= before[q];
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
  } else {
    assert sorted[p] == before[p];
    assert before[p] <= before[q];
  }
  assert sorted[q] == before[q];
}

assert forall k | 0 <= k < j + 1 :: sorted[k] <= sorted[j] by {
  if before[j] < before[j - 1] {
    assert sorted[j] == before[j - 1];
    if k == j {
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert before[j] < before[j - 1];
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
    }
  } else {
    assert before[j - 1] <= before[j];
    assert sorted[j] == before[j];
    if k == j {
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
      assert before[k] <= before[j];
    }
  }
}

j := j + 1;
```

```
    }

    assert j == i;
    assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

    assert forall p, q | i - 1 <= p < q < n :: sorted[p] <= sorted[q] by {
      if p == i - 1 {
        if q < i {
          assert q == i - 1;
        } else {
          assert i <= q < n;
          assert 0 <= i - 1 < i <= q < n;
          assert sorted[i - 1] <= sorted[q];
        }
      } else {
        assert i <= p < q < n;
        assert sorted[p] <= sorted[q];
      }
    }

    assert forall p, q | 0 <= p < i - 1 <= q < n :: sorted[p] <= sorted[q] by {
      if q == i - 1 {
        assert 0 <= p < i;
        assert sorted[p] <= sorted[i - 1];
      } else {
        assert i <= q < n;
        assert 0 <= p < i <= q < n;
        assert sorted[p] <= sorted[q];
      }
    }

    i := i - 1;
  }

  assert i == 0;
  assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_7_bubble_sort_array/program.dfy(34,20): Error: type seq<int> does not
    ↪ have a member Length
   |
34 | assert before.Length == n;
   |  ^^^^^^

problem_data/p_14_7_bubble_sort_array/program.dfy(49,23): Error: type seq<int> does not
    ↪ have a member Length
```

```
   |
49 | assert swapped.Length == before.Length;
   | ^^^^^^

problem_data/p_14_7_bubble_sort_array/program.dfy(49,40): Error: type seq<int> does not
    ↪ have a member Length
   |
49 | assert swapped.Length == before.Length;
   | ^^^^^^

problem_data/p_14_7_bubble_sort_array/program.dfy(51,13): Error: unresolved identifier: k
   |
51 | if k < j - 1 {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(52,26): Error: unresolved identifier: k
   |
52 | assert sorted[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(52,39): Error: unresolved identifier: k
   |
52 | assert sorted[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(53,27): Error: unresolved identifier: k
   |
53 | assert swapped[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(53,40): Error: unresolved identifier: k
   |
53 | assert swapped[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(54,20): Error: unresolved identifier: k
   |
54 | } else if k == j - 1 {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(55,26): Error: unresolved identifier: k
   |
55 | assert sorted[k] == before[j];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(56,27): Error: unresolved identifier: k
   |
56 | assert swapped[k] == before[j];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(57,20): Error: unresolved identifier: k
```

```
   |
57 | } else if k == j {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(58,26): Error: unresolved identifier: k
   |
58 | assert sorted[k] == before[j - 1];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(59,27): Error: unresolved identifier: k
   |
59 | assert swapped[k] == before[j - 1];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(61,28): Error: unresolved identifier: k
   |
61 | assert j + 1 <= k < sorted.Length;
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(62,26): Error: unresolved identifier: k
   |
62 | assert sorted[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(62,39): Error: unresolved identifier: k
   |
62 | assert sorted[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(63,27): Error: unresolved identifier: k
   |
63 | assert swapped[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(63,40): Error: unresolved identifier: k
   |
63 | assert swapped[k] == before[k];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(66,26): Error: type seq<?> does not
    ↪ have a member Length
   |
66 | assert sorted[..].Length == swapped.Length;
   | ^^^^^^

problem_data/p_14_7_bubble_sort_array/program.dfy(66,44): Error: type seq<int> does not
    ↪ have a member Length
   |
66 | assert sorted[..].Length == swapped.Length;
   | ^^^^^^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(67,47): Error: type seq<?> does not
    ↪ have a member Length
   |
67 | assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
   | ^^^^^^

problem_data/p_14_7_bubble_sort_array/program.dfy(95,22): Error: unresolved identifier: p
   |
95 | assert sorted[p] == before[p];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(95,35): Error: unresolved identifier: p
   |
95 | assert sorted[p] == before[p];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(96,22): Error: unresolved identifier: q
   |
96 | assert sorted[q] == before[q];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(96,35): Error: unresolved identifier: q
   |
96 | assert sorted[q] == before[q];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(97,22): Error: unresolved identifier: p
   |
97 | assert before[p] <= before[q];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(97,35): Error: unresolved identifier: q
   |
97 | assert before[p] <= before[q];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(102,13): Error: unresolved identifier:
    ↪ p
    |
102 | if p == j - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(103,26): Error: unresolved identifier:
    ↪ p
    |
103 | assert sorted[p] == before[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(104,39): Error: unresolved identifier:
    ↪ q
    |
```

```
104 | assert before[j] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(105,20): Error: unresolved identifier:
    ↪ p
    |
105 | } else if p == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(106,26): Error: unresolved identifier:
    ↪ p
    |
106 | assert sorted[p] == before[j - 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(107,43): Error: unresolved identifier:
    ↪ q
    |
107 | assert before[j - 1] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(109,26): Error: unresolved identifier:
    ↪ p
    |
109 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(109,39): Error: unresolved identifier:
    ↪ p
    |
109 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(110,26): Error: unresolved identifier:
    ↪ p
    |
110 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(110,39): Error: unresolved identifier:
    ↪ q
    |
110 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(113,24): Error: unresolved identifier:
    ↪ p
    |
113 | assert sorted[p] == before[p];
    | ^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(113,37): Error: unresolved identifier:
    ↪ p
    |
113 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,24): Error: unresolved identifier:
    ↪ p
    |
114 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,37): Error: unresolved identifier:
    ↪ q
    |
114 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(116,22): Error: unresolved identifier:
    ↪ q
    |
116 | assert sorted[q] == before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(116,35): Error: unresolved identifier:
    ↪ q
    |
116 | assert sorted[q] == before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(122,13): Error: unresolved identifier:
    ↪ k
    |
122 | if k == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(123,20): Error: unresolved identifier:
    ↪ k
    |
123 | } else if k == j - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(124,26): Error: unresolved identifier:
    ↪ k
    |
124 | assert sorted[k] == before[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(127,24): Error: unresolved identifier:
    ↪ k
    |
```

```
127 | assert 0 <= k < j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(128,26): Error: unresolved identifier:
    ↪ k
    |
128 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(128,39): Error: unresolved identifier:
    ↪ k
    |
128 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(129,26): Error: unresolved identifier:
    ↪ k
    |
129 | assert before[k] <= before[j - 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(134,13): Error: unresolved identifier:
    ↪ k
    |
134 | if k == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(136,24): Error: unresolved identifier:
    ↪ k
    |
136 | assert 0 <= k < j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(137,26): Error: unresolved identifier:
    ↪ k
    |
137 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(137,39): Error: unresolved identifier:
    ↪ k
    |
137 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(138,26): Error: unresolved identifier:
    ↪ k
    |
138 | assert before[k] <= before[j - 1];
    | ^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(139,26): Error: unresolved identifier:
    ↪ k
    |
139 | assert before[k] <= before[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(151,9): Error: unresolved identifier: p
    |
151 | if p == i - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(152,11): Error: unresolved identifier:
    ↪ q
    |
152 | if q < i {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(153,17): Error: unresolved identifier:
    ↪ q
    |
153 | assert q == i - 1;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(155,22): Error: unresolved identifier:
    ↪ q
    |
155 | assert i <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(156,35): Error: unresolved identifier:
    ↪ q
    |
156 | assert 0 <= i - 1 < i <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(157,41): Error: unresolved identifier:
    ↪ q
    |
157 | assert sorted[i - 1] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(160,20): Error: unresolved identifier:
    ↪ p
    |
160 | assert i <= p < q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(160,24): Error: unresolved identifier:
    ↪ q
    |
160 | assert i <= p < q < n;
```

```
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(161,22): Error: unresolved identifier:
    ↪ p
    |
161 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(161,35): Error: unresolved identifier:
    ↪ q
    |
161 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(166,9): Error: unresolved identifier: q
    |
166 | if q == i - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(167,20): Error: unresolved identifier:
    ↪ p
    |
167 | assert 0 <= p < i;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(168,22): Error: unresolved identifier:
    ↪ p
    |
168 | assert sorted[p] <= sorted[i - 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(170,20): Error: unresolved identifier:
    ↪ q
    |
170 | assert i <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(171,20): Error: unresolved identifier:
    ↪ p
    |
171 | assert 0 <= p < i <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(171,29): Error: unresolved identifier:
    ↪ q
    |
171 | assert 0 <= p < i <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(172,22): Error: unresolved identifier:
    ↪ p
```

```
    |
172 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(172,35): Error: unresolved identifier:
    ↪ q
    |
172 | assert sorted[p] <= sorted[q];
    | ^

75 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQJq6YlgecxuSUFbqZK1e3Aa3YOA
Model: gpt-5.5-2026-04-23
Created at: 1780671614
Finish reason: stop
Usage: 11139 tokens in, and 3470 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
```

```
  decreases i - j
{
  ghost var before := sorted[..];
  assert before == sorted[..];
  assert |before| == n;
  assert multiset(before) == multiset(inputs[..]);
  assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
  assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

  if before[j] < before[j - 1] {
    var t := sorted[j - 1];
    sorted[j - 1] := sorted[j];
    sorted[j] := t;

    assert sorted[j - 1] == before[j];
    assert sorted[j] == before[j - 1];

    ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
        ↪ 1..];
    assert |swapped| == |before|;
    forall k | 0 <= k < sorted.Length
      ensures sorted[k] == swapped[k]
    {
      if k < j - 1 {
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert swapped[k] == before[j];
      } else if k == j {
        assert sorted[k] == before[j - 1];
        assert swapped[k] == before[j - 1];
      } else {
        assert j + 1 <= k < sorted.Length;
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      }
    }
    assert |sorted[..]| == |swapped|;
    assert forall k :: 0 <= k < |sorted[..]| ==> sorted[..][k] == swapped[k];
    assert sorted[..] == swapped;

    calc {
      multiset(sorted[..]);
    ==
      multiset(swapped);
    ==
      multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
```

```
        ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
        ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
    == {
      assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
        ↪ 1..];
    }
      multiset(before);
    }
  } else {
    assert sorted[..] == before;
    assert multiset(sorted[..]) == multiset(before);
  }

  assert multiset(sorted[..]) == multiset(inputs[..]);

  forall p, q | i <= p < q < n
    ensures sorted[p] <= sorted[q]
  {
    assert sorted[p] == before[p];
    assert sorted[q] == before[q];
    assert before[p] <= before[q];
  }

  forall p, q | 0 <= p < i <= q < n
    ensures sorted[p] <= sorted[q]
  {
    if before[j] < before[j - 1] {
      if p == j - 1 {
        assert sorted[p] == before[j];
        assert before[j] <= before[q];
      } else if p == j {
        assert sorted[p] == before[j - 1];
        assert before[j - 1] <= before[q];
      } else {
        assert sorted[p] == before[p];
        assert before[p] <= before[q];
      }
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
    assert sorted[q] == before[q];
  }

  forall k | 0 <= k < j + 1
    ensures sorted[k] <= sorted[j]
  {
```

```
    if before[j] < before[j - 1] {
      assert sorted[j] == before[j - 1];
      if k == j {
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert before[j] < before[j - 1];
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
      }
    } else {
      assert before[j - 1] <= before[j];
      assert sorted[j] == before[j];
      if k == j {
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
        assert before[k] <= before[j];
      }
    }
  }

  j := j + 1;
}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

forall p, q | i - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == i - 1 {
    if q < i {
      assert q == i - 1;
    } else {
      assert i <= q < n;
      assert 0 <= i - 1 < i <= q < n;
      assert sorted[i - 1] <= sorted[q];
    }
  } else {
    assert i <= p < q < n;
    assert sorted[p] <= sorted[q];
  }
}

forall p, q | 0 <= p < i - 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if q == i - 1 {
```

```
      assert 0 <= p < i;
      assert sorted[p] <= sorted[i - 1];
    } else {
      assert i <= q < n;
      assert 0 <= p < i <= q < n;
      assert sorted[p] <= sorted[q];
    }
  }

  i := i - 1;
 }

 assert i == 0;
 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_7_bubble_sort_array/program.dfy(42,22): Error: assignment to array
    ↪ element is not allowed in this context, because the statement is in a ghost
    ↪ context; e.g., it may be guarded by a specification-only expression
   |
42 | sorted[j - 1] := sorted[j];
   | ^^

problem_data/p_14_7_bubble_sort_array/program.dfy(43,18): Error: assignment to array
    ↪ element is not allowed in this context, because the statement is in a ghost
    ↪ context; e.g., it may be guarded by a specification-only expression
   |
43 | sorted[j] := t;
   | ^^

2 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQL3ZKwdCwOk3GPmNzN8LwdddU0f
Model: gpt-5.5-2026-04-23
Created at: 1780671689
Finish reason: length
Usage: 10037 tokens in, and 859 tokens out
The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)
       ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
       ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
      decreases i - j
    {
      ghost var before := sorted[..];
      assert before == sorted[..];
      assert |before| == n;
      assert multiset(before) == multiset(inputs[..]);
      assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
      assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
      assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

      if before[j] < before[j - 1] {
        var t := sorted[j - 1];
        sorted[j - 1] := sorted[j];
        sorted[j] := t;

        assert sorted[j - 1] == before[j];
        assert sorted[j] == before[j - 1];
```

```
    ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
        ↪ 1..];
    assert |swapped| == |before|;
    forall k | 0 <= k < sorted.Length
      ensures sorted[k] == swapped[k]
    {
      if k < j - 1 {
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert swapped[k] == before[j];
      } else if k == j {
        assert sorted[k] == before[j - 1];
        assert swapped[k] == before[j - 1];
      } else {
        assert j + 1 <= k < sorted.Length;
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      }
    }
    assert |sorted[..]| == |swapped|;
    assert forall k :: 0 <= k < |sorted[..]| ==> sorted[..][k] == swapped[k];
    assert sorted[..] == swapped;

    calc {
      multiset(sorted[..]);
    ==
      multiset(swapped);
    ==
      multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
          ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
          ↪ multiset(before[j + 1..]);
    ==
      multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
    == {
      assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
          ↪ 1..];
    }
      multiset(before);
    }
  } else {
    assert sorted[..] == before;
    assert multiset(sorted[..]) == multiset(before);
  }

  assert multiset(sorted[..]) == multiset(inputs[..]);
```

```
forall p, q | i <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  assert sorted[p] == before[p];
  assert sorted[q] == before[q];
  assert before[p] <= before[q];
}

forall p, q | 0 <= p < i <= q < n
  ensures sorted[p] <= sorted[q]
{
  if before[j] < before[j - 1] {
    if p == j - 1 {
      assert sorted[p] == before[j];
      assert before[j] <= before[q];
    } else if p == j {
      assert sorted[p] == before[j - 1];
      assert before[j - 1] <= before[q];
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
  } else {
    assert sorted[p] == before[p];
    assert before[p] <= before[q];
  }
  assert sorted[q] == before[q];
}

forall k | 0 <= k < j + 1
  ensures sorted[k] <= sorted[j]
{
  if before[j] < before[j - 1] {
    assert sorted[j] == before[j - 1];
    if k == j {
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert before[j] < before[j - 1];
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
    }
  } else {
    assert before[j - 1] <= before[j];
    assert sorted[j] == before[j];
    if k == j {
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
```

```
          assert before[k] <= before[j - 1];
          assert before[k] <= before[j];
        }
      }
    }

    j := j + 1;
  }

  assert j == i;
  assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

  forall p, q | i - 1 <= p < q < n
    ensures sorted[p] <= sorted[q]
  {
    if p == i - 1 {
      if q < i {
        assert q == i - 1;
      } else {
        assert i <= q < n;
        assert 0 <= i - 1 < i <= q < n;
        assert sorted[i - 1] <= sorted[q];
      }
    } else {
      assert i <= p < q < n;
      assert sorted[p] <= sorted[q];
    }
  }

  forall p, q | 0 <= p < i - 1 <= q < n
    ensures sorted[p] <= sorted[q]
  {
    if q == i - 1 {
      assert 0 <= p < i;
      assert sorted[p] <= sorted[i - 1];
    } else {
      assert i <= q < n;
      assert 0 <= p < i <= q < n;
      assert sorted[p] <= sorted[q];
    }
  }

  i := i - 1;
 }

 assert i == 0;
 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 23990
Output tokens: 16047
Reasoning tokens: 10203
Sum of 'total tokens': 40037

Experiment Timings

Overall Experiment started at 1780671369432, ended at 1780671712254, lasting 342822ms (342.82 seconds)
Iteration #4 started at 1780671688694, ended at 1780671712253, lasting 23559ms (23.56 seconds)
Iteration #1 started at 1780671369446, ended at 1780671543250, lasting 173804ms (173.80 seconds)
Iteration #2 started at 1780671543250, ended at 1780671613783, lasting 70533ms (70.53 seconds)
Iteration #3 started at 1780671613783, ended at 1780671688694, lasting 74911ms (74.91 seconds)

Experiment 'p_14_4_merge_sort_lexicographic_array' Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY
Bad responses: 1
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 3

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_array
Natural language statement: null
Method signature: p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted: array<string>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted: array<
    ↪ string>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures
    ↪ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPzCB20U6Q0wRRt5kE7Mug0ixGmw
Model: gpt-5.5-2026-04-23
Created at: 1780670334
Finish reason: stop
Usage: 316 tokens in, and 8894 tokens out

```
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] < x
```

```
  invariant 0 <= k <= |res|
  invariant forall t: int :: 0 <= t < k ==> res[t] < x
  invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
  decreases |res| - k
{
  k := k + 1;
}

assert forall t: int :: 0 <= t < k ==> res[t] <= x by {
  forall t: int | 0 <= t < k
    ensures res[t] <= x
  {
    assert res[t] < x;
  }
}

assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert !(res[k] < x);
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
```

```
        if a < k {
          assert newRes[a] == res[a];
          assert a < b - 1;
          assert res[a] <= res[b - 1];
        } else if a == k {
          assert newRes[a] == x;
          assert k <= b - 1 < |res|;
          assert x <= res[b - 1];
        } else {
          assert k < a;
          assert newRes[a] == res[a - 1];
          assert 0 <= a - 1 < b - 1 < |res|;
          assert res[a - 1] <= res[b - 1];
        }
      }
    }
  }

  assert res == res[..k] + res[k..];
  assert multiset(newRes) == multiset(res) + multiset([x]);
  assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
  assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

  res := newRes;
 }

 assert |rest| == 0;
 assert rest == [];
 assert multiset(res) == multiset(inputs[..]);

 sorted := new string[|res|](i => res[i]);
 assert sorted.Length == |res|;
 assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
 assert sorted[..] == res;

 assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
     ↪ by {
   forall i: int, j: int | 0 <= i < j < sorted.Length
     ensures sorted[i] <= sorted[j]
   {
     assert sorted[i] == res[i];
     assert sorted[j] == res[j];
     assert res[i] <= res[j];
   }
 }

 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(45,8): Error: assertion
    ↪ might not hold
   |
45 | assert x <= res[k];
   | ^^^^^^

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(104,38): Error: index out
    ↪ of range
    |
104 | sorted := new string[|res|](i => res[i]);
    | ^


Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQ2eQyPD1jqXaF69ODznuHZK2I6R
Model: gpt-5.5-2026-04-23
Created at: 1780670548
Finish reason: stop
Usage: 1723 tokens in, and 7223 tokens out

```
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x
      invariant 0 <= k <= |res|
      invariant forall t: int :: 0 <= t < k ==> res[t] <= x
```

```
  invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
  decreases |res| - k
{
  assert res[k] <= x;
  k := k + 1;
}

assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    if res[k] <= x {
      assert k < |res| && res[k] <= x;
      assert false;
    }
    assert !(res[k] <= x);
    assert x <= res[k] || res[k] <= x;
    if !(x <= res[k]) {
      assert res[k] <= x;
      assert false;
    }
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
```

```
        if a < k {
          assert newRes[a] == res[a];
          assert a < b - 1;
          assert res[a] <= res[b - 1];
        } else if a == k {
          assert newRes[a] == x;
          assert k <= b - 1 < |res|;
          assert x <= res[b - 1];
        } else {
          assert k < a;
          assert newRes[a] == res[a - 1];
          assert 0 <= a - 1 < b - 1 < |res|;
          assert res[a - 1] <= res[b - 1];
        }
      }
    }
  }

  assert res == res[..k] + res[k..];
  assert multiset(newRes) == multiset(res) + multiset([x]);
  assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
  assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

  res := newRes;
}

assert |rest| == 0;
assert rest == [];
assert multiset(res) == multiset(inputs[..]);

sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
assert sorted.Length == |res|;
assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
  forall i: int | 0 <= i < |res|
    ensures sorted[..][i] == res[i]
  {
    assert sorted[..][i] == sorted[i];
    assert sorted[i] == res[i];
  }
}
assert sorted[..] == res;

assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
    ↪ by {
  forall i: int, j: int | 0 <= i < j < sorted.Length
    ensures sorted[i] <= sorted[j]
  {
    assert sorted[i] == res[i];
    assert sorted[j] == res[j];
    assert res[i] <= res[j];
```

```
    }
  }

  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(42,8): Error: assertion
    ↪ might not hold
   |
42 | assert x <= res[k] || res[k] <= x;
   |        ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQ5r9dlMa34K7uchXHUNU9LCUUz5
Model: gpt-5.5-2026-04-23
Created at: 1780670747
Finish reason: stop
Usage: 3122 tokens in, and 7605 tokens out

```
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x
```

```
  invariant 0 <= k <= |res|
  invariant forall t: int :: 0 <= t < k ==> res[t] <= x
  invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
  decreases |res| - k
{
  assert res[k] <= x;
  k := k + 1;
}

if k < |res| {
  assert !(res[k] <= x);
  var y := res[k];
  var n := if |x| <= |y| then |x| else |y|;
  assert n <= |x| && n <= |y|;
  var p := 0;
  while p < n && x[p] == y[p]
    invariant 0 <= p <= n
    invariant n <= |x| && n <= |y|
    invariant forall q: int :: 0 <= q < p ==> x[q] == y[q]
    decreases n - p
  {
    p := p + 1;
  }
  if p == n {
    if |x| <= |y| {
      assert n == |x|;
      assert forall q: int :: 0 <= q < |x| ==> x[q] == y[q];
      assert x <= y;
    } else {
      assert n == |y|;
      assert forall q: int :: 0 <= q < |y| ==> y[q] == x[q];
      assert y <= x;
    }
  } else {
    assert p < n;
    assert p < |x| && p < |y|;
    assert x[p] != y[p];
    if x[p] < y[p] {
      assert x <= y;
    } else {
      assert y[p] < x[p];
      assert y <= x;
    }
  }
  assert x <= y || y <= x;
  assert y == res[k];
  assert x <= res[k] || res[k] <= x;
  assert x <= res[k];
}
assert k < |res| ==> x <= res[k];
```

```
assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
      if a < k {
        assert newRes[a] == res[a];
        assert a < b - 1;
        assert res[a] <= res[b - 1];
      } else if a == k {
        assert newRes[a] == x;
        assert k <= b - 1 < |res|;
        assert x <= res[b - 1];
      } else {
        assert k < a;
        assert newRes[a] == res[a - 1];
        assert 0 <= a - 1 < b - 1 < |res|;
        assert res[a - 1] <= res[b - 1];
      }
    }
  }
}
```

```
    assert res == res[..k] + res[k..];
    assert multiset(newRes) == multiset(res) + multiset([x]);
    assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
    assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

    res := newRes;
  }

  assert |rest| == 0;
  assert rest == [];
  assert multiset(res) == multiset(inputs[..]);

  sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
  assert sorted.Length == |res|;
  assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
  assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
    forall i: int | 0 <= i < |res|
      ensures sorted[..][i] == res[i]
    {
      assert sorted[..][i] == sorted[i];
      assert sorted[i] == res[i];
    }
  }
  assert sorted[..] == res;

  assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
      ↪ by {
    forall i: int, j: int | 0 <= i < j < sorted.Length
      ensures sorted[i] <= sorted[j]
    {
      assert sorted[i] == res[i];
      assert sorted[j] == res[j];
      assert res[i] <= res[j];
    }
  }

  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(61,10): Error: assertion
    ↪ might not hold
   |
61 | assert x <= y;
   |        ^^^^^^
```

```
problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(64,10): Error: assertion
    ↪ might not hold
   |
64 | assert y <= x;
   | ^^^^^^


Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQ9UXXjGPifhvkLsDi2BTrIqX5gS
Model: gpt-5.5-2026-04-23
Created at: 1780670972
Finish reason: length
Usage: 4965 tokens in, and 1117 tokens out
The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted:
    ↪ array<string>)
       ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
       ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x
```

```
  invariant 0 <= k <= |res|
  invariant forall t: int :: 0 <= t < k ==> res[t] <= x
  invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
  decreases |res| - k
{
  assert res[k] <= x;
  k := k + 1;
}

if k < |res| {
  assert !(res[k] <= x);
  var y := res[k];
  var n := if |x| <= |y| then |x| else |y|;
  assert n <= |x| && n <= |y|;
  var p := 0;
  while p < n && x[p] == y[p]
    invariant 0 <= p <= n
    invariant n <= |x| && n <= |y|
    invariant forall q: int :: 0 <= q < p ==> x[q] == y[q]
    decreases n - p
  {
    p := p + 1;
  }
  if p == n {
    if |x| <= |y| {
      assert n == |x|;
      assert forall q: int :: 0 <= q < |x| ==> x[q] == y[q];
      assert x <= y;
    } else {
      assert n == |y|;
      assert forall q: int :: 0 <= q < |y| ==> y[q] == x[q];
      assert y <= x;
    }
  } else {
    assert p < n;
    assert p < |x| && p < |y|;
    assert x[p] != y[p];
    if x[p] < y[p] {
      assert x <= y;
    } else {
      assert y[p] < x[p];
      assert y <= x;
    }
  }
  assert x <= y || y <= x;
  assert y == res[k];
  assert x <= res[k] || res[k] <= x;
  assert x <= res[k];
}
assert k < |res| ==> x <= res[k];
```

```
assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
      if a < k {
        assert newRes[a] == res[a];
        assert a < b - 1;
        assert res[a] <= res[b - 1];
      } else if a == k {
        assert newRes[a] == x;
        assert k <= b - 1 < |res|;
        assert x <= res[b - 1];
      } else {
        assert k < a;
        assert newRes[a] == res[a - 1];
        assert 0 <= a - 1 < b - 1 < |res|;
        assert res[a - 1] <= res[b - 1];
      }
    }
  }
}
```

```
    assert res == res[..k] + res[k..];
    assert multiset(newRes) == multiset(res) + multiset([x]);
    assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
    assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

    res := newRes;
  }

  assert |rest| == 0;
  assert rest == [];
  assert multiset(res) == multiset(inputs[..]);

  sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
  assert sorted.Length == |res|;
  assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
  assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
    forall i: int | 0 <= i < |res|
      ensures sorted[..][i] == res[i]
    {
      assert sorted[..][i] == sorted[i];
      assert sorted[i] == res[i];
    }
  }
  assert sorted[..] == res;

  assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
      ↪ by {
    forall i: int, j: int | 0 <= i < j < sorted.Length
      ensures sorted[i] <= sorted[j]
    {
      assert sorted[i] == res[i];
      assert sorted[j] == res[j];
      assert res[i] <= res[j];
    }
  }

  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 10126
Output tokens: 24839
Reasoning tokens: 20505
Sum of 'total tokens': 34965

Experiment Timings

Overall Experiment started at 1780670333069, ended at 1780671004410, lasting 671341ms (671.34 seconds)
Iteration #4 started at 1780670969587, ended at 1780671004410, lasting 34823ms (34.82 seconds)
Iteration #1 started at 1780670333084, ended at 1780670546703, lasting 213619ms (213.62 seconds)
Iteration #2 started at 1780670546707, ended at 1780670742811, lasting 196104ms (196.10 seconds)
Iteration #3 started at 1780670742811, ended at 1780670969586, lasting 226775ms (226.78 seconds)

Experiment 'p_14_4_merge_sort_lexicographic_seq' Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY
Bad responses: 1
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 4
Hard fails (resolution): 2
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_seq
Natural language statement: null
Method signature: p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
    ↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPpadx3N7055LnejUh4czMKJfBWe
Model: gpt-5.5-2026-04-23
Created at: 1780669738
Finish reason: stop
Usage: 314 tokens in, and 2289 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  {
    var x := inputs[n];
    var old := sorted;
    var k := 0;

    while k < |old| && old[k] <= x
      invariant 0 <= k <= |old|
      invariant old == sorted
      invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
      invariant forall i :: 0 <= i < k ==> old[i] <= x
    {
      k := k + 1;
    }
```

```
    if k < |old| {
      assert !(old[k] <= x);
      assert x <= old[k];
    }

    sorted := old[..k] + [x] + old[k..];

    assert forall i, j | 0 <= i < j < |sorted| ensures sorted[i] <= sorted[j]
    {
      if j < k {
        assert sorted[i] == old[i];
        assert sorted[j] == old[j];
      } else if j == k {
        assert i < k;
        assert sorted[i] == old[i];
        assert sorted[j] == x;
      } else if i < k {
        assert k < j;
        assert sorted[i] == old[i];
        assert sorted[j] == old[j - 1];
        assert 0 <= i < j - 1 < |old|;
      } else if i == k {
        assert sorted[i] == x;
        assert sorted[j] == old[j - 1];
        assert k <= j - 1 < |old|;
        if k < |old| {
          assert x <= old[k];
          if k < j - 1 {
            assert old[k] <= old[j - 1];
          }
        }
      } else {
        assert k < i;
        assert sorted[i] == old[i - 1];
        assert sorted[j] == old[j - 1];
        assert 0 <= i - 1 < j - 1 < |old|;
      }
    }

    assert old == old[..k] + old[k..];
    assert sorted == old[..k] + [x] + old[k..];
    assert multiset(sorted) == multiset(old[..k]) + multiset([x]) + multiset(old[k..]);
    assert multiset(old) == multiset(old[..k]) + multiset(old[k..]);
    assert multiset(sorted) == multiset(old) + multiset([x]);
    assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
    assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

    n := n + 1;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(13,8): Error: invalid Ident
   |
13 | var old := sorted;
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(16,18): Error: openparen
    ↪ expected
   |
16 | while k < |old| && old[k] <= x
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(16,26): Error: openparen
    ↪ expected
   |
16 | while k < |old| && old[k] <= x
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(17,6): Error: closeparen
    ↪ expected
   |
17 | invariant 0 <= k <= |old|
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(17,30): Error: openparen
    ↪ expected
   |
17 | invariant 0 <= k <= |old|
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(18,20): Error: openparen
    ↪ expected
   |
18 | invariant old == sorted
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,6): Error: closeparen
    ↪ expected
   |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,48): Error: openparen
    ↪ expected
   |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
   | ^
```

```
problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,57): Error: openparen
    ↪ expected
   |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,67): Error: openparen
    ↪ expected
   |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(20,6): Error: closeparen
    ↪ expected
   |
20 | invariant forall i :: 0 <= i < k ==> old[i] <= x
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(20,46): Error: openparen
    ↪ expected
   |
20 | invariant forall i :: 0 <= i < k ==> old[i] <= x
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(21,4): Error: closeparen
    ↪ expected
   |
21 | {
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(25,15): Error: openparen
    ↪ expected
   |
25 | if k < |old| {
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(26,18): Error: openparen
    ↪ expected
   |
26 | assert !(old[k] <= x);
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(26,27): Error: closeparen
    ↪ expected
   |
26 | assert !(old[k] <= x);
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,21): Error: openparen
    ↪ expected
   |
```

```
27 | assert x <= old[k];
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,24): Error: closeparen
    ↪ expected
   |
27 | assert x <= old[k];
   | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(30,17): Error: openparen
    ↪ expected
   |
30 | sorted := old[..k] + [x] + old[k..];
   | ^

19 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPqTDa8hMLXyDrqdSJj5tXzS9jnl
Model: gpt-5.5-2026-04-23
Created at: 1780669793
Finish reason: stop
Usage: 2985 tokens in, and 2289 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  {
    var x := inputs[n];
    var prev := sorted;
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
      invariant prev == sorted
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
      invariant forall i :: 0 <= i < k ==> prev[i] <= x
    {
      k := k + 1;
    }
```

```
    if k < |prev| {
      assert !(prev[k] <= x);
      assert x <= prev[k];
    }

    sorted := prev[..k] + [x] + prev[k..];

    assert forall i, j | 0 <= i < j < |sorted| ensures sorted[i] <= sorted[j]
    {
      if j < k {
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j];
      } else if j == k {
        assert i < k;
        assert sorted[i] == prev[i];
        assert sorted[j] == x;
      } else if i < k {
        assert k < j;
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i < j - 1 < |prev|;
      } else if i == k {
        assert sorted[i] == x;
        assert sorted[j] == prev[j - 1];
        assert k <= j - 1 < |prev|;
        if k < |prev| {
          assert x <= prev[k];
          if k < j - 1 {
            assert prev[k] <= prev[j - 1];
          }
        }
      } else {
        assert k < i;
        assert sorted[i] == prev[i - 1];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i - 1 < j - 1 < |prev|;
      }
    }

    assert prev == prev[..k] + prev[k..];
    assert sorted == prev[..k] + [x] + prev[k..];
    assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
    assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
    assert multiset(sorted) == multiset(prev) + multiset([x]);
    assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
    assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

    n := n + 1;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(72,6): Error: invalid
    ↪ AssertStmt
   |
72 | n := n + 1;
   | ^

1 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPrO8YITgjCUWYUUqvSLigqjil2i
Model: gpt-5.5-2026-04-23
Created at: 1780669850
Finish reason: stop
Usage: 2939 tokens in, and 1505 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  {
    var x := inputs[n];
    var prev := sorted;
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
      invariant prev == sorted
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
      invariant forall i :: 0 <= i < k ==> prev[i] <= x
    {
      k := k + 1;
    }

    if k < |prev| {
      assert !(prev[k] <= x);
      assert x <= prev[k];
    }

    sorted := prev[..k] + [x] + prev[k..];
```

```
    forall i, j | 0 <= i < j < |sorted|
      ensures sorted[i] <= sorted[j]
    {
      if j < k {
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j];
      } else if j == k {
        assert i < k;
        assert sorted[i] == prev[i];
        assert sorted[j] == x;
      } else if i < k {
        assert k < j;
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i < j - 1 < |prev|;
      } else if i == k {
        assert sorted[i] == x;
        assert sorted[j] == prev[j - 1];
        assert k <= j - 1 < |prev|;
        if k < |prev| {
          assert x <= prev[k];
          if k < j - 1 {
            assert prev[k] <= prev[j - 1];
          }
        } else {
          assert false;
        }
      } else {
        assert k < i;
        assert sorted[i] == prev[i - 1];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i - 1 < j - 1 < |prev|;
      }
    }

    assert prev == prev[..k] + prev[k..];
    assert sorted == prev[..k] + [x] + prev[k..];
    assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
    assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
    assert multiset(sorted) == multiset(prev) + multiset([x]);
    assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
    assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

    n := n + 1;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:
```

```
problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(4,0): Error: a postcondition
    ↪  could not be proved on this return path
  |
4 | {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(3,26): Related location:
    ↪ this is the postcondition that could not be proved
  |
3 |  ensures multiset(inputs) == multiset(sorted)
  | ^^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,6): Error: assertion
    ↪ might not hold
   |
27 | assert x <= prev[k];
   | ^^^^^^


Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPsVpwTvMmEKdkFYp33JY8VWxDis
Model: gpt-5.5-2026-04-23
Created at: 1780669919
Finish reason: stop
Usage: 4035 tokens in, and 8296 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    decreases |inputs| - n
  {
    var x := inputs[n];
    var prev := sorted;
    assert multiset(prev) == multiset(inputs[..n]);
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
```

```
  invariant prev == sorted
  invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
  invariant forall i :: 0 <= i < k ==> prev[i] <= x
  decreases |prev| - k
{
  k := k + 1;
}

if k < |prev| {
  assert !(prev[k] <= x);
  assert x <= prev[k] || prev[k] <= x;
  assert x <= prev[k];
}

sorted := prev[..k] + [x] + prev[k..];

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j];
  } else if j == k {
    assert i < k;
    assert sorted[i] == prev[i];
    assert sorted[j] == x;
  } else if i < k {
    assert k < j;
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i < j - 1 < |prev|;
  } else if i == k {
    assert sorted[i] == x;
    assert sorted[j] == prev[j - 1];
    assert k <= j - 1 < |prev|;
    assert k < |prev|;
    assert x <= prev[k];
    if k < j - 1 {
      assert prev[k] <= prev[j - 1];
    }
  } else {
    assert k < i;
    assert sorted[i] == prev[i - 1];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i - 1 < j - 1 < |prev|;
  }
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
```

```
    assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
    assert multiset(sorted) == multiset(prev) + multiset([x]);
    assert x == inputs[n];
    assert multiset(sorted) == multiset(inputs[..n]) + multiset([inputs[n]]);
    assert inputs[..(n + 1)] == inputs[..n] + [inputs[n]];
    assert multiset(inputs[..(n + 1)]) == multiset(inputs[..n]) + multiset([inputs[n]]);
    assert multiset(sorted) == multiset(inputs[..(n + 1)]);

    n := n + 1;
  }

  assert n == |inputs|;
  assert inputs[..n] == inputs;
  assert multiset(inputs) == multiset(sorted);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(30,6): Error: assertion
    ↪ might not hold
   |
30 | assert x <= prev[k] || prev[k] <= x;
   |  ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPwfzU4K8cYSrfS5IgWqlA4I6FJs
Model: gpt-5.5-2026-04-23
Created at: 1780670177
Finish reason: length
Usage: 4948 tokens in, and 5348 tokens out
The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<
    ↪ string>)
```

```
      ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    decreases |inputs| - n
  {
    var x := inputs[n];
    var prev := sorted;
    assert multiset(prev) == multiset(inputs[..n]);
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
      invariant prev == sorted
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
      invariant forall i :: 0 <= i < k ==> prev[i] <= x
      decreases |prev| - k
    {
      k := k + 1;
    }

    if k < |prev| {
      assert !(prev[k] <= x);
      assert x <= prev[k] || prev[k] <= x;
      assert x <= prev[k];
    }

    sorted := prev[..k] + [x] + prev[k..];

    forall i, j | 0 <= i < j < |sorted|
      ensures sorted[i] <= sorted[j]
    {
      if j < k {
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j];
      } else if j == k {
        assert i < k;
        assert sorted[i] == prev[i];
        assert sorted[j] == x;
      } else if i < k {
        assert k < j;
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i < j - 1 < |prev|;
      } else if i == k {
        assert sorted[i] == x;
```

```
      assert sorted[j] == prev[j - 1];
      assert k <= j - 1 < |prev|;
      assert k < |prev|;
      assert x <= prev[k];
      if k < j - 1 {
        assert prev[k] <= prev[j - 1];
      }
    } else {
      assert k < i;
      assert sorted[i] == prev[i - 1];
      assert sorted[j] == prev[j - 1];
      assert 0 <= i - 1 < j - 1 < |prev|;
    }
  }

  assert prev == prev[..k] + prev[k..];
  assert sorted == prev[..k] + [x] + prev[k..];
  assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
  assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
  assert multiset(sorted) == multiset(prev) + multiset([x]);
  assert x == inputs[n];
  assert multiset(sorted) == multiset(inputs[..n]) + multiset([inputs[n]]);
  assert inputs[..(n + 1)] == inputs[..n] + [inputs[n]];
  assert multiset(inputs[..(n + 1)]) == multiset(inputs[..n]) + multiset([inputs[n]]);
  assert multiset(sorted) == multiset(inputs[..(n + 1)]);

  n := n + 1;
 }

 assert n == |inputs|;
 assert inputs[..n] == inputs;
 assert multiset(inputs) == multiset(sorted);
}
```

Total Token Usage

Input tokens: 15221
Output tokens: 19727
Reasoning tokens: 16509
Sum of 'total tokens': 34948

Experiment Timings

Overall Experiment started at 1780669738465, ended at 1780670325100, lasting 586635ms (586.64 seconds)
Iteration #4 started at 1780669917767, ended at 1780670159571, lasting 241804ms (241.80 seconds)
Iteration #5 started at 1780670159572, ended at 1780670325099, lasting 165527ms (165.53 seconds)
Iteration #1 started at 1780669738466, ended at 1780669791997, lasting 53531ms (53.53 seconds)
Iteration #2 started at 1780669791998, ended at 1780669849799, lasting 57801ms (57.80 seconds)
Iteration #3 started at 1780669849799, ended at 1780669917766, lasting 67967ms (67.97 seconds)

Experiment 'p_14_1_selection_sort_desc_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_14_1_selection_sort_desc_array
Natural language statement: null
Method signature: p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j], ensures
    ↪ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPczkKoTfcvgWGy3LctJj6XOB0gr
Model: gpt-5.5-2026-04-23
Created at: 1780668957
Finish reason: stop
Usage: 310 tokens in, and 9827 tokens out

```
{
  sorted := new int[inputs.Length](k => inputs[k]);
  assert forall k :: 0 <= k < inputs.Length ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var n := sorted.Length;
  var i := 0;
  while i < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> sorted[a] >= sorted[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] >= sorted[b]
    invariant multiset(sorted[..]) == multiset(inputs[..])
    decreases n - i
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
```

```
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant i < n
  invariant i <= maxIdx < j <= n
  invariant forall k :: i <= k < j ==> sorted[maxIdx] >= sorted[k]
  invariant forall a, b :: 0 <= a < b < i ==> sorted[a] >= sorted[b]
  invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] >= sorted[b]
  invariant multiset(sorted[..]) == multiset(inputs[..])
  decreases n - j
{
  if sorted[j] > sorted[maxIdx] {
    maxIdx := j;
  }
  j := j + 1;
}

assert j == n;
ghost var prev := sorted[..];
assert |prev| == n;
assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];
assert forall a, b :: 0 <= a < b < i ==> prev[a] >= prev[b];
assert forall a, b :: 0 <= a < i <= b < n ==> prev[a] >= prev[b];

var tmp := sorted[i];
sorted[i] := sorted[maxIdx];
sorted[maxIdx] := tmp;

assert sorted[i] == prev[maxIdx];
assert sorted[maxIdx] == prev[i];
assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> sorted[k] == prev[k];

forall k | 0 <= k < n
  ensures sorted[k] == (prev[i := prev[maxIdx]][maxIdx := prev[i]])[k]
{
  if k == i {
    if maxIdx == i {
      assert (prev[i := prev[maxIdx]][maxIdx := prev[i]])[k] == prev[i];
    } else {
      assert (prev[i := prev[maxIdx]][maxIdx := prev[i]])[k] == prev[maxIdx];
    }
  } else if k == maxIdx {
    assert (prev[i := prev[maxIdx]][maxIdx := prev[i]])[k] == prev[i];
  } else {
    assert (prev[i := prev[maxIdx]][maxIdx := prev[i]])[k] == prev[k];
  }
}
assert |sorted[..]| == |prev[i := prev[maxIdx]][maxIdx := prev[i]]|;
assert sorted[..] == prev[i := prev[maxIdx]][maxIdx := prev[i]];

if i == maxIdx {
  assert prev[i := prev[maxIdx]][maxIdx := prev[i]] == prev;
```

```
  } else {
    assert multiset(prev[i := prev[maxIdx]]) == multiset(prev) - multiset{prev[i]} +
        ↪ multiset{prev[maxIdx]};
    assert (prev[i := prev[maxIdx]])[maxIdx] == prev[maxIdx];
    assert multiset(prev[i := prev[maxIdx]][maxIdx := prev[i]]) ==
          multiset(prev[i := prev[maxIdx]]) - multiset{prev[maxIdx]} + multiset{prev[i
              ↪ ]};
    assert multiset(prev[i := prev[maxIdx]][maxIdx := prev[i]]) == multiset(prev);
  }
  assert multiset(sorted[..]) == multiset(prev);
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var next := i + 1;

  forall a, b | 0 <= a < b < next
    ensures sorted[a] >= sorted[b]
  {
    if b < i {
      assert sorted[a] == prev[a];
      assert sorted[b] == prev[b];
      assert prev[a] >= prev[b];
    } else {
      assert b == i;
      assert sorted[a] == prev[a];
      assert sorted[b] == prev[maxIdx];
      assert prev[a] >= prev[maxIdx];
    }
  }

  forall a, b | 0 <= a < next <= b < n
    ensures sorted[a] >= sorted[b]
  {
    if a < i {
      assert sorted[a] == prev[a];
      if b == maxIdx {
        assert sorted[b] == prev[i];
        assert prev[a] >= prev[i];
      } else {
        assert sorted[b] == prev[b];
        assert prev[a] >= prev[b];
      }
    } else {
      assert a == i;
      assert sorted[a] == prev[maxIdx];
      if b == maxIdx {
        assert sorted[b] == prev[i];
        assert prev[maxIdx] >= prev[i];
      } else {
        assert sorted[b] == prev[b];
        assert prev[maxIdx] >= prev[b];
      }
```

```
      }
    }

    i := next;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_1_selection_sort_desc_array/program.dfy(5,46): Error: index out of
    ↪ range
  |
5 | sorted := new int[inputs.Length](k => inputs[k]);
  |                                              ^

problem_data/p_14_1_selection_sort_desc_array/program.dfy(5,46): Error: insufficient
    ↪ reads clause to read array element; Consider extracting inputs[k] to a local
    ↪ variable before the lambda expression, or adding 'reads inputs' in the enclosing
    ↪ lambda specification for resolution
  |
5 | sorted := new int[inputs.Length](k => inputs[k]);
  |                                              ^


Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPgg7Y34IWBlKNDo5NeUr8R2c9X2
Model: gpt-5.5-2026-04-23
Created at: 1780669186
Finish reason: stop
Usage: 1935 tokens in, and 6376 tokens out

```
{
  var s := inputs[..];
  var n := |s|;

  var i := 0;
  while i < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
```

```
  invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
  invariant multiset(s) == multiset(inputs[..])
  decreases n - i
{
  var maxIdx := i;
  var j := i + 1;
  while j < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i < n
    invariant i <= maxIdx < j <= n
    invariant forall k :: i <= k < j ==> s[maxIdx] >= s[k]
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - j
  {
    if s[j] > s[maxIdx] {
      maxIdx := j;
    }
    j := j + 1;
  }

  assert j == n;
  var prev := s;
  var vi := prev[i];
  var vmax := prev[maxIdx];
  assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];

  s := prev[i := vmax][maxIdx := vi];
  assert |s| == n;

  if maxIdx == i {
    assert vi == vmax;
    assert prev[i := vmax] == prev;
    assert s == prev[i := vi];
    assert prev[i := vi] == prev;
    assert s == prev;
  } else {
    assert i < maxIdx;
    var mid := prev[i := vmax];
    assert s == mid[maxIdx := vi];
    assert mid[maxIdx] == vmax;

    if vi == vmax {
      assert mid == prev;
      assert s == prev[maxIdx := vi];
      assert prev[maxIdx := vi] == prev;
      assert s == prev;
    } else {
      assert multiset(mid) == multiset(prev) - multiset{vi} + multiset{vmax};
```

```
    assert multiset(s) == multiset(mid) - multiset{vmax} + multiset{vi};
    assert multiset(s) == multiset(prev);
  }

  assert s[i] == prev[maxIdx];
  assert s[maxIdx] == prev[i];
  forall k | 0 <= k < n && k != i && k != maxIdx
    ensures s[k] == prev[k]
  {
    assert mid[k] == prev[k];
    assert s[k] == mid[k];
  }
}

assert s[i] == prev[maxIdx];
assert s[maxIdx] == prev[i];
assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> s[k] == prev[k];
assert multiset(s) == multiset(prev);
assert multiset(s) == multiset(inputs[..]);

var next := i + 1;

forall a, b | 0 <= a < b < next
  ensures s[a] >= s[b]
{
  if b < i {
    assert a != i && a != maxIdx;
    assert b != i && b != maxIdx;
    assert s[a] == prev[a];
    assert s[b] == prev[b];
    assert prev[a] >= prev[b];
  } else {
    assert b == i;
    assert a < i;
    assert a != i && a != maxIdx;
    assert s[a] == prev[a];
    assert s[b] == prev[maxIdx];
    assert prev[a] >= prev[maxIdx];
  }
}

forall a, b | 0 <= a < next <= b < n
  ensures s[a] >= s[b]
{
  if a < i {
    assert a != i && a != maxIdx;
    assert s[a] == prev[a];
    if b == maxIdx {
      assert s[b] == prev[i];
      assert prev[a] >= prev[i];
    } else {
```

```
        assert b != i;
        assert s[b] == prev[b];
        assert prev[a] >= prev[b];
      }
    } else {
      assert a == i;
      assert s[a] == prev[maxIdx];
      if b == maxIdx {
        assert s[b] == prev[i];
        assert prev[maxIdx] >= prev[i];
      } else {
        assert b != i;
        assert s[b] == prev[b];
        assert prev[maxIdx] >= prev[b];
      }
    }
  }

  i := next;
}

assert i == n;
sorted := new int[n];
assert sorted.Length == n;
assert sorted != inputs;

var k := 0;
while k < n
  invariant sorted.Length == n
  invariant sorted != inputs
  invariant |s| == n
  invariant i == n
  invariant 0 <= k <= n
  invariant forall t :: 0 <= t < k ==> sorted[t] == s[t]
  invariant forall a, b :: 0 <= a < b < n ==> s[a] >= s[b]
  invariant multiset(s) == multiset(inputs[..])
  decreases n - k
{
  sorted[k] := s[k];
  k := k + 1;
}

assert forall t :: 0 <= t < n ==> sorted[t] == s[t];
assert |sorted[..]| == |s|;
forall t | 0 <= t < |s|
  ensures sorted[..][t] == s[t]
{
  assert sorted[..][t] == sorted[t];
  assert sorted[t] == s[t];
}
assert sorted[..] == s;
```

```
  forall a, b | 0 <= a < b < sorted.Length
    ensures sorted[a] >= sorted[b]
  {
    assert sorted.Length == n;
    assert sorted[a] == s[a];
    assert sorted[b] == s[b];
    assert s[a] >= s[b];
  }

  assert multiset(sorted[..]) == multiset(s);
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Final Program

```
method p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)
       ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]
       ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var s := inputs[..];
  var n := |s|;

  var i := 0;
  while i < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - i
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
      invariant |s| == n
      invariant n == inputs.Length
      invariant 0 <= i < n
      invariant i <= maxIdx < j <= n
      invariant forall k :: i <= k < j ==> s[maxIdx] >= s[k]
      invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
      invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
      invariant multiset(s) == multiset(inputs[..])
      decreases n - j
    {
      if s[j] > s[maxIdx] {
        maxIdx := j;
      }
      j := j + 1;
```

```
  }

  assert j == n;
  var prev := s;
  var vi := prev[i];
  var vmax := prev[maxIdx];
  assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];

  s := prev[i := vmax][maxIdx := vi];
  assert |s| == n;

  if maxIdx == i {
    assert vi == vmax;
    assert prev[i := vmax] == prev;
    assert s == prev[i := vi];
    assert prev[i := vi] == prev;
    assert s == prev;
  } else {
    assert i < maxIdx;
    var mid := prev[i := vmax];
    assert s == mid[maxIdx := vi];
    assert mid[maxIdx] == vmax;

    if vi == vmax {
      assert mid == prev;
      assert s == prev[maxIdx := vi];
      assert prev[maxIdx := vi] == prev;
      assert s == prev;
    } else {
      assert multiset(mid) == multiset(prev) - multiset{vi} + multiset{vmax};
      assert multiset(s) == multiset(mid) - multiset{vmax} + multiset{vi};
      assert multiset(s) == multiset(prev);
    }

    assert s[i] == prev[maxIdx];
    assert s[maxIdx] == prev[i];
    forall k | 0 <= k < n && k != i && k != maxIdx
      ensures s[k] == prev[k]
    {
      assert mid[k] == prev[k];
      assert s[k] == mid[k];
    }
  }

  assert s[i] == prev[maxIdx];
  assert s[maxIdx] == prev[i];
  assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> s[k] == prev[k];
  assert multiset(s) == multiset(prev);
  assert multiset(s) == multiset(inputs[..]);

  var next := i + 1;
```

```
  forall a, b | 0 <= a < b < next
    ensures s[a] >= s[b]
  {
    if b < i {
      assert a != i && a != maxIdx;
      assert b != i && b != maxIdx;
      assert s[a] == prev[a];
      assert s[b] == prev[b];
      assert prev[a] >= prev[b];
    } else {
      assert b == i;
      assert a < i;
      assert a != i && a != maxIdx;
      assert s[a] == prev[a];
      assert s[b] == prev[maxIdx];
      assert prev[a] >= prev[maxIdx];
    }
  }

  forall a, b | 0 <= a < next <= b < n
    ensures s[a] >= s[b]
  {
    if a < i {
      assert a != i && a != maxIdx;
      assert s[a] == prev[a];
      if b == maxIdx {
        assert s[b] == prev[i];
        assert prev[a] >= prev[i];
      } else {
        assert b != i;
        assert s[b] == prev[b];
        assert prev[a] >= prev[b];
      }
    } else {
      assert a == i;
      assert s[a] == prev[maxIdx];
      if b == maxIdx {
        assert s[b] == prev[i];
        assert prev[maxIdx] >= prev[i];
      } else {
        assert b != i;
        assert s[b] == prev[b];
        assert prev[maxIdx] >= prev[b];
      }
    }
  }

  i := next;
}
```

```
  assert i == n;
  sorted := new int[n];
  assert sorted.Length == n;
  assert sorted != inputs;

  var k := 0;
  while k < n
    invariant sorted.Length == n
    invariant sorted != inputs
    invariant |s| == n
    invariant i == n
    invariant 0 <= k <= n
    invariant forall t :: 0 <= t < k ==> sorted[t] == s[t]
    invariant forall a, b :: 0 <= a < b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - k
  {
    sorted[k] := s[k];
    k := k + 1;
  }

  assert forall t :: 0 <= t < n ==> sorted[t] == s[t];
  assert |sorted[..]| == |s|;
  forall t | 0 <= t < |s|
    ensures sorted[..][t] == s[t]
  {
    assert sorted[..][t] == sorted[t];
    assert sorted[t] == s[t];
  }
  assert sorted[..] == s;

  forall a, b | 0 <= a < b < sorted.Length
    ensures sorted[a] >= sorted[b]
  {
    assert sorted.Length == n;
    assert sorted[a] == s[a];
    assert sorted[b] == s[b];
    assert s[a] >= s[b];
  }

  assert multiset(sorted[..]) == multiset(s);
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 2245
Output tokens: 16203
Reasoning tokens: 13130
Sum of 'total tokens': 18448

Experiment Timings

Overall Experiment started at 1780668956393, ended at 1780669345399, lasting 389006ms (389.01 seconds)
Iteration #1 started at 1780668956395, ended at 1780669185759, lasting 229364ms (229.36 seconds)
Iteration #2 started at 1780669185812, ended at 1780669345399, lasting 159587ms (159.59 seconds)

Experiment 'p_14_1_selection_sort_desc_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_1_selection_sort_desc_seq
Natural language statement: null
Method signature: p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j], ensures multiset
    ↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPaVrHXsMzQAYBxTsOHPEArSM3aa
Model: gpt-5.5-2026-04-23
Created at: 1780668803
Finish reason: stop
Usage: 308 tokens in, and 7362 tokens out

```
{
  sorted := [];
  var rest := inputs;

  while |rest| > 0
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
    invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] >= rest[j]
    decreases |rest|
  {
    var oldSorted := sorted;
    var oldRest := rest;
    assert |oldRest| > 0;
    assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] >= oldSorted[j];
    assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] >=
        ↪ oldRest[j];

    var maxIdx := 0;
    var k := 1;
    while k < |oldRest|
```

```
  invariant 1 <= k <= |oldRest|
  invariant 0 <= maxIdx < k
  invariant forall t :: 0 <= t < k ==> oldRest[maxIdx] >= oldRest[t]
  decreases |oldRest| - k
{
  var kk := k;
  var prevMaxIdx := maxIdx;
  if oldRest[kk] > oldRest[maxIdx] {
    maxIdx := kk;
    forall t | 0 <= t < kk
      ensures oldRest[maxIdx] >= oldRest[t]
    {
      assert oldRest[prevMaxIdx] >= oldRest[t];
      assert oldRest[maxIdx] > oldRest[prevMaxIdx];
    }
    assert oldRest[maxIdx] >= oldRest[kk];
  } else {
    forall t | 0 <= t < kk
      ensures oldRest[maxIdx] >= oldRest[t]
    {
    }
    assert oldRest[maxIdx] >= oldRest[kk];
  }
  forall t | 0 <= t < kk + 1
    ensures oldRest[maxIdx] >= oldRest[t]
  {
    if t < kk {
      assert oldRest[maxIdx] >= oldRest[t];
    } else {
      assert t == kk;
      assert oldRest[maxIdx] >= oldRest[kk];
    }
  }
  k := kk + 1;
}

assert k == |oldRest|;
assert forall t :: 0 <= t < |oldRest| ==> oldRest[maxIdx] >= oldRest[t];

var x := oldRest[maxIdx];
var prefix := oldRest[..maxIdx];
var suffix := oldRest[maxIdx + 1..];
var newRest := prefix + suffix;

assert |prefix| == maxIdx;
assert |suffix| == |oldRest| - (maxIdx + 1);
assert oldRest == prefix + [x] + suffix;
assert newRest == prefix + suffix;
calc {
  multiset(oldRest);
==
```

```
  multiset(prefix + [x] + suffix);
==
  multiset(prefix + [x]) + multiset(suffix);
==
  multiset(prefix) + multiset([x]) + multiset(suffix);
==
  multiset(prefix) + multiset(suffix) + multiset([x]);
==
  multiset(prefix + suffix) + multiset([x]);
==
  multiset(newRest) + multiset([x]);
}

sorted := oldSorted + [x];
rest := newRest;

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] >= sorted[j]
{
  if j < |oldSorted| {
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] >= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == x;
    assert oldSorted[i] >= oldRest[maxIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] >= rest[j]
{
  var oldIndex: int;
  if j < maxIdx {
    oldIndex := j;
    assert j < |prefix|;
    assert rest[j] == prefix[j];
    assert prefix[j] == oldRest[j];
    assert rest[j] == oldRest[oldIndex];
  } else {
    oldIndex := j + 1;
    assert maxIdx <= j;
    assert j + 1 < |oldRest|;
    assert 0 <= j - maxIdx;
```

```
      assert j - maxIdx < |suffix|;
      assert rest[j] == suffix[j - maxIdx];
      assert suffix[j - maxIdx] == oldRest[maxIdx + 1 + (j - maxIdx)];
      assert maxIdx + 1 + (j - maxIdx) == j + 1;
      assert rest[j] == oldRest[oldIndex];
    }
    assert 0 <= oldIndex < |oldRest|;
    if i < |oldSorted| {
      assert sorted[i] == oldSorted[i];
      assert oldSorted[i] >= oldRest[oldIndex];
    } else {
      assert i == |oldSorted|;
      assert sorted[i] == x;
      assert x >= oldRest[oldIndex];
    }
  }

  assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
  calc {
    multiset(sorted) + multiset(rest);
  ==
    (multiset(oldSorted) + multiset([x])) + multiset(newRest);
  ==
    multiset(oldSorted) + (multiset(newRest) + multiset([x]));
  ==
    multiset(oldSorted) + multiset(oldRest);
  ==
    multiset(inputs);
  }
  assert multiset(inputs) == multiset(sorted) + multiset(rest);
 }

 assert rest == [];
 assert multiset(rest) == multiset([]);
 assert multiset(sorted) + multiset(rest) == multiset(sorted);
 assert multiset(inputs) == multiset(sorted);
}
```

Final Program

```
method p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)
      ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
      ensures multiset(inputs) == multiset(sorted)
{
 sorted := [];
 var rest := inputs;

 while |rest| > 0
   invariant multiset(inputs) == multiset(sorted) + multiset(rest)
   invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
```

```
  invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] >= rest[j]
  decreases |rest|
{
  var oldSorted := sorted;
  var oldRest := rest;
  assert |oldRest| > 0;
  assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] >= oldSorted[j];
  assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] >=
      ↪ oldRest[j];

  var maxIdx := 0;
  var k := 1;
  while k < |oldRest|
    invariant 1 <= k <= |oldRest|
    invariant 0 <= maxIdx < k
    invariant forall t :: 0 <= t < k ==> oldRest[maxIdx] >= oldRest[t]
    decreases |oldRest| - k
  {
    var kk := k;
    var prevMaxIdx := maxIdx;
    if oldRest[kk] > oldRest[maxIdx] {
      maxIdx := kk;
      forall t | 0 <= t < kk
        ensures oldRest[maxIdx] >= oldRest[t]
      {
        assert oldRest[prevMaxIdx] >= oldRest[t];
        assert oldRest[maxIdx] > oldRest[prevMaxIdx];
      }
      assert oldRest[maxIdx] >= oldRest[kk];
    } else {
      forall t | 0 <= t < kk
        ensures oldRest[maxIdx] >= oldRest[t]
      {
      }
      assert oldRest[maxIdx] >= oldRest[kk];
    }
    forall t | 0 <= t < kk + 1
      ensures oldRest[maxIdx] >= oldRest[t]
    {
      if t < kk {
        assert oldRest[maxIdx] >= oldRest[t];
      } else {
        assert t == kk;
        assert oldRest[maxIdx] >= oldRest[kk];
      }
    }
    k := kk + 1;
  }

  assert k == |oldRest|;
  assert forall t :: 0 <= t < |oldRest| ==> oldRest[maxIdx] >= oldRest[t];
```

```
var x := oldRest[maxIdx];
var prefix := oldRest[..maxIdx];
var suffix := oldRest[maxIdx + 1..];
var newRest := prefix + suffix;

assert |prefix| == maxIdx;
assert |suffix| == |oldRest| - (maxIdx + 1);
assert oldRest == prefix + [x] + suffix;
assert newRest == prefix + suffix;
calc {
  multiset(oldRest);
==
  multiset(prefix + [x] + suffix);
==
  multiset(prefix + [x]) + multiset(suffix);
==
  multiset(prefix) + multiset([x]) + multiset(suffix);
==
  multiset(prefix) + multiset(suffix) + multiset([x]);
==
  multiset(prefix + suffix) + multiset([x]);
==
  multiset(newRest) + multiset([x]);
}

sorted := oldSorted + [x];
rest := newRest;

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] >= sorted[j]
{
  if j < |oldSorted| {
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] >= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == x;
    assert oldSorted[i] >= oldRest[maxIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] >= rest[j]
```

```
    {
      var oldIndex: int;
      if j < maxIdx {
        oldIndex := j;
        assert j < |prefix|;
        assert rest[j] == prefix[j];
        assert prefix[j] == oldRest[j];
        assert rest[j] == oldRest[oldIndex];
      } else {
        oldIndex := j + 1;
        assert maxIdx <= j;
        assert j + 1 < |oldRest|;
        assert 0 <= j - maxIdx;
        assert j - maxIdx < |suffix|;
        assert rest[j] == suffix[j - maxIdx];
        assert suffix[j - maxIdx] == oldRest[maxIdx + 1 + (j - maxIdx)];
        assert maxIdx + 1 + (j - maxIdx) == j + 1;
        assert rest[j] == oldRest[oldIndex];
      }
      assert 0 <= oldIndex < |oldRest|;
      if i < |oldSorted| {
        assert sorted[i] == oldSorted[i];
        assert oldSorted[i] >= oldRest[oldIndex];
      } else {
        assert i == |oldSorted|;
        assert sorted[i] == x;
        assert x >= oldRest[oldIndex];
      }
    }

    assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
    calc {
      multiset(sorted) + multiset(rest);
    ==
      (multiset(oldSorted) + multiset([x])) + multiset(newRest);
    ==
      multiset(oldSorted) + (multiset(newRest) + multiset([x]));
    ==
      multiset(oldSorted) + multiset(oldRest);
    ==
      multiset(inputs);
    }
    assert multiset(inputs) == multiset(sorted) + multiset(rest);
  }

  assert rest == [];
  assert multiset(rest) == multiset([]);
  assert multiset(sorted) + multiset(rest) == multiset(sorted);
  assert multiset(inputs) == multiset(sorted);
}
```

Total Token Usage

Input tokens: 308
Output tokens: 7362
Reasoning tokens: 5725
Sum of 'total tokens': 7670

Experiment Timings

Overall Experiment started at 1780668800185, ended at 1780668953966, lasting 153781ms (153.78 seconds)
Iteration #1 started at 1780668800196, ended at 1780668953966, lasting 153770ms (153.77 seconds)

Experiment 'p_14_2_selection_sort_interval_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_array
Natural language statement: null
Method signature: p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

Requires

- `forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted:
    ↪ array<int>)

The method should respect the following contract:

requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound, ensures forall i
    ↪ , j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures multiset(
    ↪ inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPlqZF3KGimF7UX4jZnMnUpMZvjb
Model: gpt-5.5-2026-04-23
Created at: 1780669506
Finish reason: stop
Usage: 358 tokens in, and 10107 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
  {
    sorted[k] := inputs[k];
    k := k + 1;
  }
  assert forall t :: 0 <= t < n ==> sorted[t] == inputs[t];
  assert sorted[..] == inputs[..];

  var i := 0;
```

```
while i < n
  invariant 0 <= i <= n
  invariant n == inputs.Length
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i + 1 <= j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    invariant forall t :: i <= t < j ==> sorted[min] <= sorted[t]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert sorted[j] < sorted[oldMin];
      min := j;
      assert forall t :: i <= t < j ==> sorted[min] <= sorted[t] by {
        forall t | i <= t < j
          ensures sorted[min] <= sorted[t]
        {
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[t];
        }
      }
      assert sorted[min] <= sorted[j];
    } else {
      assert sorted[min] <= sorted[j];
    }
    assert forall t :: i <= t < j + 1 ==> sorted[min] <= sorted[t] by {
      forall t | i <= t < j + 1
        ensures sorted[min] <= sorted[t]
      {
        if t == j {
          assert sorted[min] <= sorted[j];
        } else {
          assert i <= t < j;
          assert sorted[min] <= sorted[t];
        }
      }
    }
    j := j + 1;
  }

  ghost var before := sorted[..];
```

```
assert |before| == n;
assert forall p, q :: 0 <= p < q < i ==> before[p] <= before[q];
assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];
assert forall t :: i <= t < n ==> before[min] <= before[t];

if min != i {
  assert i < min < n;
  sorted[i], sorted[min] := sorted[min], sorted[i];
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert sorted[..] == before[i := before[min]][min := before[i]];
  assert multiset(before[i := before[min]]) == multiset(before) - multiset{before[i]}
      ↪ + multiset{before[min]};
  assert multiset(before[i := before[min]][min := before[i]]) == multiset(before[i :=
      ↪ before[min]]) - multiset{before[min]} + multiset{before[i]};
  assert multiset(sorted[..]) == multiset(before);
} else {
  assert sorted[..] == before;
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert multiset(sorted[..]) == multiset(before);
}
assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall p, q :: 0 <= p < q < i + 1 ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert p != i && p != min;
      assert q != i && q != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[q];
      assert before[p] <= before[q];
    } else {
      assert q == i;
      assert p < i;
      assert p != i && p != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[min];
      assert before[p] <= before[min];
    }
  }
}

assert forall p, q :: 0 <= p < i + 1 <= q < n ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < i + 1 <= q < n
    ensures sorted[p] <= sorted[q]
  {
```

```
      if p < i {
        assert p != i && p != min;
        assert sorted[p] == before[p];
        if q == min {
          assert sorted[q] == before[i];
          assert before[p] <= before[i];
        } else {
          assert q != i;
          assert sorted[q] == before[q];
          assert before[p] <= before[q];
        }
      } else {
        assert p == i;
        assert sorted[p] == before[min];
        if q == min {
          assert sorted[q] == before[i];
          assert before[min] <= before[i];
        } else {
          assert q != i;
          assert sorted[q] == before[q];
          assert before[min] <= before[q];
        }
      }
    }
  }

  i := i + 1;
 }

 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Final Program

```
method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
    ↪ sorted: array<int>)
      requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound
      ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
      ensures multiset(inputs[..]) == multiset(sorted[..])
{
 var n := inputs.Length;
 sorted := new int[n];

 var k := 0;
 while k < n
   invariant 0 <= k <= n
   invariant n == inputs.Length
   invariant sorted.Length == n
   invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
```

```
{
  sorted[k] := inputs[k];
  k := k + 1;
}
assert forall t :: 0 <= t < n ==> sorted[t] == inputs[t];
assert sorted[..] == inputs[..];

var i := 0;
while i < n
  invariant 0 <= i <= n
  invariant n == inputs.Length
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i + 1 <= j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    invariant forall t :: i <= t < j ==> sorted[min] <= sorted[t]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert sorted[j] < sorted[oldMin];
      min := j;
      assert forall t :: i <= t < j ==> sorted[min] <= sorted[t] by {
        forall t | i <= t < j
          ensures sorted[min] <= sorted[t]
        {
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[t];
        }
      }
      assert sorted[min] <= sorted[j];
    } else {
      assert sorted[min] <= sorted[j];
    }
    assert forall t :: i <= t < j + 1 ==> sorted[min] <= sorted[t] by {
      forall t | i <= t < j + 1
        ensures sorted[min] <= sorted[t]
      {
        if t == j {
          assert sorted[min] <= sorted[j];
        } else {
          assert i <= t < j;
```

```
        assert sorted[min] <= sorted[t];
      }
    }
  }
  j := j + 1;
}

ghost var before := sorted[..];
assert |before| == n;
assert forall p, q :: 0 <= p < q < i ==> before[p] <= before[q];
assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];
assert forall t :: i <= t < n ==> before[min] <= before[t];

if min != i {
  assert i < min < n;
  sorted[i], sorted[min] := sorted[min], sorted[i];
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert sorted[..] == before[i := before[min]][min := before[i]];
  assert multiset(before[i := before[min]]) == multiset(before) - multiset{before[i]}
      ↪ + multiset{before[min]};
  assert multiset(before[i := before[min]][min := before[i]]) == multiset(before[i :=
      ↪ before[min]]) - multiset{before[min]} + multiset{before[i]};
  assert multiset(sorted[..]) == multiset(before);
} else {
  assert sorted[..] == before;
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert multiset(sorted[..]) == multiset(before);
}
assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall p, q :: 0 <= p < q < i + 1 ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert p != i && p != min;
      assert q != i && q != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[q];
      assert before[p] <= before[q];
    } else {
      assert q == i;
      assert p < i;
      assert p != i && p != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[min];
      assert before[p] <= before[min];
```

```
      }
    }
  }

  assert forall p, q :: 0 <= p < i + 1 <= q < n ==> sorted[p] <= sorted[q] by {
    forall p, q | 0 <= p < i + 1 <= q < n
      ensures sorted[p] <= sorted[q]
    {
      if p < i {
        assert p != i && p != min;
        assert sorted[p] == before[p];
        if q == min {
          assert sorted[q] == before[i];
          assert before[p] <= before[i];
        } else {
          assert q != i;
          assert sorted[q] == before[q];
          assert before[p] <= before[q];
        }
      } else {
        assert p == i;
        assert sorted[p] == before[min];
        if q == min {
          assert sorted[q] == before[i];
          assert before[min] <= before[i];
        } else {
          assert q != i;
          assert sorted[q] == before[q];
          assert before[min] <= before[q];
        }
      }
    }
  }

  i := i + 1;
 }

 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 358
Output tokens: 10107
Reasoning tokens: 8475
Sum of 'total tokens': 10465

Experiment Timings

Overall Experiment started at 1780669505281, ended at 1780669736279, lasting 230998ms (231.00 seconds)
Iteration #1 started at 1780669505282, ended at 1780669736278, lasting 230996ms (231.00 seconds)

Experiment 'p_14_2_selection_sort_interval_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_seq
Natural language statement: null
Method signature: p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

Requires

- `forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int
    ↪ >)

The method should respect the following contract:

requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound, ensures forall i, j
    ↪ :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset(inputs) ==
    ↪ multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPjNaeSagIF9yBwJgmczeyIaXgE4
Model: gpt-5.5-2026-04-23
Created at: 1780669353
Finish reason: stop
Usage: 358 tokens in, and 7439 tokens out

```
{
  sorted := [];
  var rest := inputs;
  var v: int := 0;

  while v <= bound
    invariant 0 <= v <= bound + 1
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i :: 0 <= i < |sorted| ==> sorted[i] < v
    invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
    decreases bound + 1 - v
  {
    assert 0 <= v <= bound;

    while exists k :: 0 <= k < |rest| && rest[k] == v
      invariant 0 <= v <= bound
      invariant multiset(inputs) == multiset(sorted) + multiset(rest)
```

```
  invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  invariant forall i :: 0 <= i < |sorted| ==> sorted[i] <= v
  invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
  decreases |rest|
{
  var k :| 0 <= k < |rest| && rest[k] == v;
  var oldSorted := sorted;
  var oldRest := rest;
  var newSorted := oldSorted + [v];
  var newRest := oldRest[..k] + oldRest[(k + 1)..];

  assert oldRest == oldRest[..k] + [oldRest[k]] + oldRest[(k + 1)..];
  assert oldRest[k] == v;
  assert multiset(oldRest) == multiset([v]) + multiset(newRest);
  assert multiset(newSorted) == multiset(oldSorted) + multiset([v]);
  calc {
    multiset(inputs);
    == multiset(oldSorted) + multiset(oldRest);
    == multiset(oldSorted) + (multiset([v]) + multiset(newRest));
    == (multiset(oldSorted) + multiset([v])) + multiset(newRest);
    == multiset(newSorted) + multiset(newRest);
  }

  assert forall a, b :: 0 <= a < b < |newSorted| ==> newSorted[a] <= newSorted[b] by {
    forall a, b | 0 <= a < b < |newSorted|
      ensures newSorted[a] <= newSorted[b]
    {
      if b < |oldSorted| {
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert a < |oldSorted|;
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == v;
        assert oldSorted[a] <= v;
      }
    }
  }

  assert forall t :: 0 <= t < |newSorted| ==> newSorted[t] <= v by {
    forall t | 0 <= t < |newSorted|
      ensures newSorted[t] <= v
    {
      if t < |oldSorted| {
        assert newSorted[t] == oldSorted[t];
      } else {
        assert t == |oldSorted|;
        assert newSorted[t] == v;
      }
    }
```

```
    }

    assert forall t :: 0 <= t < |newRest| ==> v <= newRest[t] <= bound by {
      forall t | 0 <= t < |newRest|
        ensures v <= newRest[t] <= bound
      {
        if t < k {
          assert newRest[t] == oldRest[t];
        } else {
          assert t + 1 < |oldRest|;
          assert newRest[t] == oldRest[t + 1];
        }
      }
    }

    sorted := newSorted;
    rest := newRest;
  }

  assert !(exists k :: 0 <= k < |rest| && rest[k] == v);
  assert forall t :: 0 <= t < |rest| ==> v + 1 <= rest[t] <= bound by {
    forall t | 0 <= t < |rest|
      ensures v + 1 <= rest[t] <= bound
    {
      assert v <= rest[t] <= bound;
      assert rest[t] != v;
      assert v < rest[t];
    }
  }
  assert forall t :: 0 <= t < |sorted| ==> sorted[t] < v + 1 by {
    forall t | 0 <= t < |sorted|
      ensures sorted[t] < v + 1
    {
      assert sorted[t] <= v;
    }
  }
  v := v + 1;
 }

 assert v == bound + 1;
 if |rest| != 0 {
   assert 0 <= 0 < |rest|;
   assert v <= rest[0] <= bound;
   assert false;
 }
 assert |rest| == 0;
 assert rest == [];
 assert multiset(rest) == multiset{};
 assert multiset(inputs) == multiset(sorted);
}
```

Final Program

```
method p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted:
    ↪ seq<int>)
      requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound
      ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var rest := inputs;
  var v: int := 0;

  while v <= bound
    invariant 0 <= v <= bound + 1
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i :: 0 <= i < |sorted| ==> sorted[i] < v
    invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
    decreases bound + 1 - v
  {
    assert 0 <= v <= bound;

    while exists k :: 0 <= k < |rest| && rest[k] == v
      invariant 0 <= v <= bound
      invariant multiset(inputs) == multiset(sorted) + multiset(rest)
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall i :: 0 <= i < |sorted| ==> sorted[i] <= v
      invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
      decreases |rest|
    {
      var k :| 0 <= k < |rest| && rest[k] == v;
      var oldSorted := sorted;
      var oldRest := rest;
      var newSorted := oldSorted + [v];
      var newRest := oldRest[..k] + oldRest[(k + 1)..];

      assert oldRest == oldRest[..k] + [oldRest[k]] + oldRest[(k + 1)..];
      assert oldRest[k] == v;
      assert multiset(oldRest) == multiset([v]) + multiset(newRest);
      assert multiset(newSorted) == multiset(oldSorted) + multiset([v]);
      calc {
        multiset(inputs);
        == multiset(oldSorted) + multiset(oldRest);
        == multiset(oldSorted) + (multiset([v]) + multiset(newRest));
        == (multiset(oldSorted) + multiset([v])) + multiset(newRest);
        == multiset(newSorted) + multiset(newRest);
      }

      assert forall a, b :: 0 <= a < b < |newSorted| ==> newSorted[a] <= newSorted[b] by {
        forall a, b | 0 <= a < b < |newSorted|
          ensures newSorted[a] <= newSorted[b]
        {
```

```
      if b < |oldSorted| {
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert a < |oldSorted|;
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == v;
        assert oldSorted[a] <= v;
      }
    }
  }

  assert forall t :: 0 <= t < |newSorted| ==> newSorted[t] <= v by {
    forall t | 0 <= t < |newSorted|
      ensures newSorted[t] <= v
    {
      if t < |oldSorted| {
        assert newSorted[t] == oldSorted[t];
      } else {
        assert t == |oldSorted|;
        assert newSorted[t] == v;
      }
    }
  }

  assert forall t :: 0 <= t < |newRest| ==> v <= newRest[t] <= bound by {
    forall t | 0 <= t < |newRest|
      ensures v <= newRest[t] <= bound
    {
      if t < k {
        assert newRest[t] == oldRest[t];
      } else {
        assert t + 1 < |oldRest|;
        assert newRest[t] == oldRest[t + 1];
      }
    }
  }

  sorted := newSorted;
  rest := newRest;
}

assert !(exists k :: 0 <= k < |rest| && rest[k] == v);
assert forall t :: 0 <= t < |rest| ==> v + 1 <= rest[t] <= bound by {
  forall t | 0 <= t < |rest|
    ensures v + 1 <= rest[t] <= bound
  {
    assert v <= rest[t] <= bound;
    assert rest[t] != v;
    assert v < rest[t];
```

```
    }
   }
   assert forall t :: 0 <= t < |sorted| ==> sorted[t] < v + 1 by {
    forall t | 0 <= t < |sorted|
      ensures sorted[t] < v + 1
    {
      assert sorted[t] <= v;
    }
   }
   v := v + 1;
  }

  assert v == bound + 1;
  if |rest| != 0 {
   assert 0 <= 0 < |rest|;
   assert v <= rest[0] <= bound;
   assert false;
  }
  assert |rest| == 0;
  assert rest == [];
  assert multiset(rest) == multiset{};
  assert multiset(inputs) == multiset(sorted);
}
```

Total Token Usage

Input tokens: 358
Output tokens: 7439
Reasoning tokens: 6143
Sum of 'total tokens': 7797

Experiment Timings

Overall Experiment started at 1780669352432, ended at 1780669500966, lasting 148534ms (148.53 seconds)
Iteration #1 started at 1780669352458, ended at 1780669500966, lasting 148508ms (148.51 seconds)

Experiment 'p_14_7_bubble_sort_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_14_7_bubble_sort_seq
Natural language statement: null
Method signature: p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
    ↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQA7UKzGFbm594FYtcj5ogjmtlbE
Model: gpt-5.5-2026-04-23
Created at: 1780671011
Finish reason: stop
Usage: 308 tokens in, and 6250 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant multiset(sorted) == multiset(inputs[..n])
  {
    var x := inputs[n];
    var k := 0;
    while k < |sorted| && sorted[k] <= x
      invariant 0 <= k <= |sorted|
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall t :: 0 <= t < k ==> sorted[t] <= x
      invariant multiset(sorted) == multiset(inputs[..n])
    {
      k := k + 1;
    }

    var old := sorted;
```

```
ghost var oldLen := |old|;
assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
assert forall t :: 0 <= t < k ==> old[t] <= x;
assert 0 <= k <= oldLen;
if k < oldLen {
  assert !(k < |old| && old[k] <= x);
  assert !(old[k] <= x);
  assert x < old[k];
}

sorted := old[..k] + [x] + old[k..];
assert |old[..k]| == k;
assert |old[..k] + [x]| == k + 1;
assert |sorted| == oldLen + 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < k {
      assert 0 <= i < j < |old|;
      assert old[i] <= old[j];
      assert sorted[i] == old[i];
      assert sorted[j] == old[j];
    } else if j == k {
      assert i < k;
      assert old[i] <= x;
      assert sorted[i] == old[i];
      assert sorted[j] == x;
    } else {
      assert k < j;
      if i < k {
        assert j - 1 < oldLen;
        assert k <= j - 1;
        assert i < j - 1;
        assert old[i] <= old[j - 1];
        assert sorted[i] == old[i];
        assert sorted[j] == old[j - 1];
      } else if i == k {
        assert j - 1 < oldLen;
        assert k <= j - 1;
        assert k < oldLen;
        assert x < old[k];
        if k < j - 1 {
          assert old[k] <= old[j - 1];
          assert x <= old[j - 1];
        } else {
          assert k == j - 1;
          assert x < old[j - 1];
        }
        assert sorted[i] == x;
```

```
          assert sorted[j] == old[j - 1];
        } else {
          assert k < i;
          assert 0 <= i - 1 < j - 1 < oldLen;
          assert old[i - 1] <= old[j - 1];
          assert sorted[i] == old[i - 1];
          assert sorted[j] == old[j - 1];
        }
      }
    }
   }

   assert old == old[..k] + old[k..];
   assert sorted == old[..k] + [x] + old[k..];
   assert multiset(old) == multiset(old[..k]) + multiset(old[k..]);
   assert multiset(sorted) == multiset(old[..k]) + multiset([x]) + multiset(old[k..]);
   assert multiset(sorted) == multiset(old) + multiset([x]);
   assert x == inputs[n];
   assert inputs[..n + 1] == inputs[..n] + [x];
   assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([x]);
   assert multiset(sorted) == multiset(inputs[..n + 1]);

   n := n + 1;
  }
  assert n == |inputs|;
  assert inputs[..n] == inputs;
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_7_bubble_sort_seq/program.dfy(23,8): Error: invalid Ident
   |
23 | var old := sorted;
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(24,28): Error: openparen expected
   |
24 | ghost var oldLen := |old|;
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(25,43): Error: openparen expected
   |
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(25,52): Error: openparen expected
   |
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
   | ^
```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(25,62): Error: openparen expected
   |
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(25,65): Error: closeparen expected
   |
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(26,41): Error: openparen expected
   |
26 | assert forall t :: 0 <= t < k ==> old[t] <= x;
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(26,49): Error: closeparen expected
   |
26 | assert forall t :: 0 <= t < k ==> old[t] <= x;
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,23): Error: openparen expected
   |
29 | assert !(k < |old| && old[k] <= x);
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,31): Error: openparen expected
   |
29 | assert !(k < |old| && old[k] <= x);
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,40): Error: verticalbar expected
   |
29 | assert !(k < |old| && old[k] <= x);
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(30,18): Error: openparen expected
   |
30 | assert !(old[k] <= x);
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(30,27): Error: closeparen expected
   |
30 | assert !(old[k] <= x);
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(31,20): Error: openparen expected
   |
31 | assert x < old[k];
   | ^
```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(31,23): Error: closeparen expected
   |
31 | assert x < old[k];
   | ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(34,17): Error: openparen expected
   |
34 | sorted := old[..k] + [x] + old[k..];
   | ^

16 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQCGPflwNTeU7uL2ScyYmnf1jKvr
Model: gpt-5.5-2026-04-23
Created at: 1780671144
Finish reason: stop
Usage: 2979 tokens in, and 2056 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant multiset(sorted) == multiset(inputs[..n])
  {
    var x := inputs[n];
    var k := 0;
    while k < |sorted| && sorted[k] <= x
      invariant 0 <= k <= |sorted|
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall t :: 0 <= t < k ==> sorted[t] <= x
      invariant multiset(sorted) == multiset(inputs[..n])
    {
      k := k + 1;
    }

    var prev := sorted;
    ghost var prevLen := |prev|;
    assert forall a, b :: 0 <= a < b < |prev| ==> prev[a] <= prev[b];
    assert forall t :: 0 <= t < k ==> prev[t] <= x;
    assert 0 <= k <= prevLen;
    if k < prevLen {
      assert !(k < |prev| && prev[k] <= x);
```

```
  assert !(prev[k] <= x);
  assert x < prev[k];
}

sorted := prev[..k] + [x] + prev[k..];
assert |prev[..k]| == k;
assert |prev[..k] + [x]| == k + 1;
assert |sorted| == prevLen + 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < k {
      assert 0 <= i < j < |prev|;
      assert prev[i] <= prev[j];
      assert sorted[i] == prev[i];
      assert sorted[j] == prev[j];
    } else if j == k {
      assert i < k;
      assert prev[i] <= x;
      assert sorted[i] == prev[i];
      assert sorted[j] == x;
    } else {
      assert k < j;
      if i < k {
        assert j - 1 < prevLen;
        assert k <= j - 1;
        assert i < j - 1;
        assert prev[i] <= prev[j - 1];
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j - 1];
      } else if i == k {
        assert j - 1 < prevLen;
        assert k <= j - 1;
        assert k < prevLen;
        assert x < prev[k];
        if k < j - 1 {
          assert prev[k] <= prev[j - 1];
          assert x <= prev[j - 1];
        } else {
          assert k == j - 1;
          assert x < prev[j - 1];
        }
        assert sorted[i] == x;
        assert sorted[j] == prev[j - 1];
      } else {
        assert k < i;
        assert 0 <= i - 1 < j - 1 < prevLen;
        assert prev[i - 1] <= prev[j - 1];
        assert sorted[i] == prev[i - 1];
```

```
          assert sorted[j] == prev[j - 1];
        }
      }
    }
  }

  assert prev == prev[..k] + prev[k..];
  assert sorted == prev[..k] + [x] + prev[k..];
  assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
  assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
  assert multiset(sorted) == multiset(prev) + multiset([x]);
  assert x == inputs[n];
  assert inputs[..n + 1] == inputs[..n] + [x];
  assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([x]);
  assert multiset(sorted) == multiset(inputs[..n + 1]);

  n := n + 1;
 }
 assert n == |inputs|;
 assert inputs[..n] == inputs;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_7_bubble_sort_seq/program.dfy(1,7): Error: Verification of '
    ↪ p_14_4_bubble_sort_seq' timed out after 31 seconds. (the limit can be increased
    ↪ using --verification-time-limit)
  |
1 | method p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)
  | ^^^^^^^^^^^^^^^^^^^^^^


Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnQDkAbwxDj77iltVxwu34SAX5eJ5
Model: gpt-5.5-2026-04-23
Created at: 1780671236
Finish reason: stop
Usage: 3545 tokens in, and 4845 tokens out

```
{
  sorted := [];
```

```
var rest := inputs;

while |rest| > 0
  invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
  invariant multiset(inputs) == multiset(sorted) + multiset(rest)
  decreases |rest|
{
  assert 0 < |rest|;
  var minIdx := 0;
  var scan := 1;
  assert forall t :: 0 <= t < 1 ==> rest[0] <= rest[t];

  while scan < |rest|
    invariant 0 <= minIdx < |rest|
    invariant 1 <= scan <= |rest|
    invariant minIdx < scan
    invariant forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t]
    decreases |rest| - scan
  {
    var prevMin := minIdx;
    if rest[scan] < rest[prevMin] {
      minIdx := scan;
      assert forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t] by {
        forall t | 0 <= t < scan
          ensures rest[minIdx] <= rest[t]
        {
          assert rest[prevMin] <= rest[t];
          assert rest[minIdx] == rest[scan];
          assert rest[scan] < rest[prevMin];
        }
      }
    } else {
      assert minIdx == prevMin;
      assert rest[minIdx] <= rest[scan];
    }

    assert forall t :: 0 <= t < scan + 1 ==> rest[minIdx] <= rest[t] by {
      forall t | 0 <= t < scan + 1
        ensures rest[minIdx] <= rest[t]
      {
        if t < scan {
          assert rest[minIdx] <= rest[t];
        } else {
          assert t == scan;
          assert rest[minIdx] <= rest[scan];
        }
      }
    }
    scan := scan + 1;
  }
```

```
assert scan == |rest|;
var m := rest[minIdx];
assert forall t :: 0 <= t < |rest| ==> m <= rest[t];

var priorSorted := sorted;
var priorRest := rest;
ghost var priorSortedLen := |priorSorted|;
ghost var priorRestLen := |priorRest|;

sorted := priorSorted + [m];
rest := priorRest[..minIdx] + priorRest[minIdx + 1..];

assert |sorted| == priorSortedLen + 1;
assert |rest| == priorRestLen - 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < priorSortedLen {
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == priorSorted[j];
      assert priorSorted[i] <= priorSorted[j];
    } else {
      assert j == priorSortedLen;
      assert i < priorSortedLen;
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == m;
      assert priorSorted[i] <= priorRest[minIdx];
    }
  }
}

assert forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j] by
    ↪ {
  forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
    ensures sorted[i] <= rest[j]
  {
    if j < minIdx {
      assert rest[j] == priorRest[j];
      if i < priorSortedLen {
        assert sorted[i] == priorSorted[i];
        assert priorSorted[i] <= priorRest[j];
      } else {
        assert i == priorSortedLen;
        assert sorted[i] == m;
        assert m <= priorRest[j];
      }
    } else {
      assert j + 1 < priorRestLen;
```

```
        assert rest[j] == priorRest[j + 1];
        if i < priorSortedLen {
          assert sorted[i] == priorSorted[i];
          assert priorSorted[i] <= priorRest[j + 1];
        } else {
          assert i == priorSortedLen;
          assert sorted[i] == m;
          assert m <= priorRest[j + 1];
        }
      }
    }
  }

  assert priorRest == priorRest[..minIdx] + [m] + priorRest[minIdx + 1..];
  assert multiset(priorRest) == multiset(priorRest[..minIdx]) + multiset([m]) + multiset
      ↪ (priorRest[minIdx + 1..]);
  assert multiset(rest) == multiset(priorRest[..minIdx]) + multiset(priorRest[minIdx +
      ↪ 1..]);
  assert multiset(priorRest) == multiset(rest) + multiset([m]);
  assert multiset(sorted) == multiset(priorSorted) + multiset([m]);
  assert multiset(inputs) == multiset(priorSorted) + multiset(priorRest);
  assert multiset(sorted) + multiset(rest) == multiset(inputs);
  assert multiset(inputs) == multiset(sorted) + multiset(rest);
 }

 assert |rest| == 0;
 assert rest == [];
 assert multiset(inputs) == multiset(sorted) + multiset(rest);
 assert multiset(inputs) == multiset(sorted);
}
```

Final Program

```
method p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)
       ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
       ensures multiset(inputs) == multiset(sorted)
{
 sorted := [];
 var rest := inputs;

 while |rest| > 0
   invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
   invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
   invariant multiset(inputs) == multiset(sorted) + multiset(rest)
   decreases |rest|
 {
   assert 0 < |rest|;
   var minIdx := 0;
   var scan := 1;
   assert forall t :: 0 <= t < 1 ==> rest[0] <= rest[t];
```

```
  while scan < |rest|
    invariant 0 <= minIdx < |rest|
    invariant 1 <= scan <= |rest|
    invariant minIdx < scan
    invariant forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t]
    decreases |rest| - scan
  {
    var prevMin := minIdx;
    if rest[scan] < rest[prevMin] {
      minIdx := scan;
      assert forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t] by {
        forall t | 0 <= t < scan
          ensures rest[minIdx] <= rest[t]
        {
          assert rest[prevMin] <= rest[t];
          assert rest[minIdx] == rest[scan];
          assert rest[scan] < rest[prevMin];
        }
      }
    } else {
      assert minIdx == prevMin;
      assert rest[minIdx] <= rest[scan];
    }

    assert forall t :: 0 <= t < scan + 1 ==> rest[minIdx] <= rest[t] by {
      forall t | 0 <= t < scan + 1
        ensures rest[minIdx] <= rest[t]
      {
        if t < scan {
          assert rest[minIdx] <= rest[t];
        } else {
          assert t == scan;
          assert rest[minIdx] <= rest[scan];
        }
      }
    }
    scan := scan + 1;
  }

  assert scan == |rest|;
  var m := rest[minIdx];
  assert forall t :: 0 <= t < |rest| ==> m <= rest[t];

  var priorSorted := sorted;
  var priorRest := rest;
  ghost var priorSortedLen := |priorSorted|;
  ghost var priorRestLen := |priorRest|;

  sorted := priorSorted + [m];
  rest := priorRest[..minIdx] + priorRest[minIdx + 1..];
```

```
  assert |sorted| == priorSortedLen + 1;
  assert |rest| == priorRestLen - 1;

  assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
    forall i, j | 0 <= i < j < |sorted|
      ensures sorted[i] <= sorted[j]
    {
      if j < priorSortedLen {
        assert sorted[i] == priorSorted[i];
        assert sorted[j] == priorSorted[j];
        assert priorSorted[i] <= priorSorted[j];
      } else {
        assert j == priorSortedLen;
        assert i < priorSortedLen;
        assert sorted[i] == priorSorted[i];
        assert sorted[j] == m;
        assert priorSorted[i] <= priorRest[minIdx];
      }
    }
  }

  assert forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j] by
      ↪ {
    forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
      ensures sorted[i] <= rest[j]
    {
      if j < minIdx {
        assert rest[j] == priorRest[j];
        if i < priorSortedLen {
          assert sorted[i] == priorSorted[i];
          assert priorSorted[i] <= priorRest[j];
        } else {
          assert i == priorSortedLen;
          assert sorted[i] == m;
          assert m <= priorRest[j];
        }
      } else {
        assert j + 1 < priorRestLen;
        assert rest[j] == priorRest[j + 1];
        if i < priorSortedLen {
          assert sorted[i] == priorSorted[i];
          assert priorSorted[i] <= priorRest[j + 1];
        } else {
          assert i == priorSortedLen;
          assert sorted[i] == m;
          assert m <= priorRest[j + 1];
        }
      }
    }
  }
```

```
    assert priorRest == priorRest[..minIdx] + [m] + priorRest[minIdx + 1..];
    assert multiset(priorRest) == multiset(priorRest[..minIdx]) + multiset([m]) + multiset
        ↪ (priorRest[minIdx + 1..]);
    assert multiset(rest) == multiset(priorRest[..minIdx]) + multiset(priorRest[minIdx +
        ↪ 1..]);
    assert multiset(priorRest) == multiset(rest) + multiset([m]);
    assert multiset(sorted) == multiset(priorSorted) + multiset([m]);
    assert multiset(inputs) == multiset(priorSorted) + multiset(priorRest);
    assert multiset(sorted) + multiset(rest) == multiset(inputs);
    assert multiset(inputs) == multiset(sorted) + multiset(rest);
  }

  assert |rest| == 0;
  assert rest == [];
  assert multiset(inputs) == multiset(sorted) + multiset(rest);
  assert multiset(inputs) == multiset(sorted);
}
```

Total Token Usage

Input tokens: 6832
Output tokens: 13151
Reasoning tokens: 9535
Sum of 'total tokens': 19983

Experiment Timings

Overall Experiment started at 1780671009638, ended at 1780671363236, lasting 353598ms (353.60 seconds)
Iteration #1 started at 1780671009843, ended at 1780671142181, lasting 132338ms (132.34 seconds)
Iteration #2 started at 1780671142182, ended at 1780671235676, lasting 93494ms (93.49 seconds)
Iteration #3 started at 1780671235689, ended at 1780671363236, lasting 127547ms (127.55 seconds)